

Artificial Intelligence

Concise Study Notes: Chapters 2, 3, 4 and 5

ENCT 351 Year III, Part II

Scope. These notes follow the syllabus and micro syllabus for Chapter 2 (Problem Solving and Search, 9 hours), Chapter 3 (Knowledge Representation and Probabilistic Reasoning, 7 hours), Chapter 4 (Machine Learning Fundamentals, 10 hours) and Chapter 5 (Neural Networks and Deep Learning, 6 hours), which together carry about 46 of the 60 theory marks. Algorithms are treated as theory only, with worked numericals in place of code. Topics marked with an asterisk in the micro syllabus as descriptive only (beam search, the AlphaGo link, knowledge graphs, application lists) are left out or kept to a line. Forty MCQs with an answer key are at the end.

Contents

	Chapter	Topics	Page
2	Problem Solving and Search	Problem formulation, state space; CSP and consistency; uninformed, informed and adversarial search; hill climbing, simulated annealing; genetic algorithm	1
3	Knowledge and Probabilistic Reasoning	Knowledge based agents, Wumpus world; KR techniques and issues; propositional and predicate logic, resolution, chaining; semantic networks and frames; Bayes networks; fuzzy logic	10
4	Machine Learning Fundamentals	Four pillars, linear algebra, optimisation; ML pipeline, bias and variance, MLE and MAP; decision tree, SVM; t-SNE; semi supervised and reinforcement learning; energy based indicators	18
5	Neural Networks and Deep Learning	Neurons, activation functions, perceptron, MLP and backpropagation; deep learning, CNN, RNN, GAN, Markov models	28
	Knowledge check	40 MCQs with answer key at the end	37

Chapter 2 Problem Solving and Search

2.1 Formal Problem Definition

Problem solving agent

A problem solving agent is a goal based agent that, given a goal, finds a *sequence of actions* that leads from the current situation to a goal state. It works in four phases: **goal formulation**, **problem formulation**, **search**, and **execution** of the solution found.

A **well defined problem** has four components.

- **Initial state:** where the agent starts.
- **Actions and transition model:** $ACTIONS(s)$ lists what can be done in state s ; $RESULT(s, a)$ (the successor function) gives the state reached. Together with the initial state they define the state space.
- **Goal test:** whether a given state is a goal state.
- **Path cost:** a numeric cost for a path, usually the sum of step costs $c(s, a, s')$.

A **solution** is a path from the initial state to a goal state; an **optimal solution** has the lowest path cost among all solutions.

State space and problem space. The *state space* is the set of all states reachable from the initial state by any sequence of actions; states are nodes and actions are directed edges. The *problem space* is the whole formulation in which the search happens: the states, the operators that change them, the initial state and the goal condition. Search is the process of exploring this space for a path from initial to goal.

Types of problem. Single state (deterministic, fully observable); sensorless or conformant (no observations); contingency (nondeterministic or partly observable, plans with branches); exploration (unknown environment, must learn it).

Graph versus tree. The state space is naturally a *graph*: it can contain cycles and several paths to the same state. Search algorithms unroll it into a *search tree* whose root is the initial state and whose children are successors. In a tree the same state may appear many times, so graph search adds an *explored set* to avoid repeats.

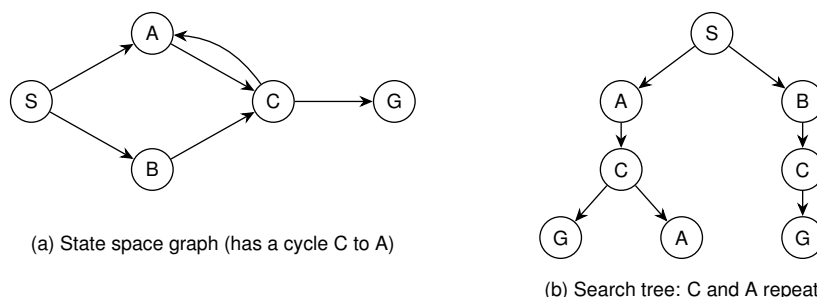


Figure 1: The same problem as a state space graph and as an unrolled search tree.

Example problems.

Problem	States	Actions	Goal test	Step cost
8 puzzle	Positions of 8 tiles and the blank on a 3×3 board ($9!/2 = 181,440$ reachable).	Move the blank up, down, left or right.	Tiles in the target order.	1 per move
Vacuum world	Agent location (2) and dirt status of each room (2^2): 8 states.	Left, Right, Suck.	No dirt in any room.	1 per action
Water jug (4 L and 3 L)	Pair (x, y) of litres in each jug.	Fill, empty, pour one jug into the other.	$x = 2$ (or $y = 2$).	1 per action

Water jug solution. Fill the 3 L jug, pour it into the 4 L jug, fill the 3 L jug again, and top up the 4 L jug from it:

$$(0, 0) \rightarrow (0, 3) \rightarrow (3, 0) \rightarrow (3, 3) \rightarrow (4, 2),$$

which leaves exactly 2 L in the 3 L jug. Pouring rule: if $x + y \geq 4$ and $y > 0$, pour 3 L into 4 L to give $(4, y - (4 - x))$.

2.2 Constraint Satisfaction Problems

Constraint satisfaction problem (CSP)

A CSP is a triple (X, D, C) : a set of variables $X = \{X_1, \dots, X_n\}$, a domain D_i of possible values for each variable, and a set of constraints C that specify allowed combinations of values. A *consistent* assignment violates no constraint; a *complete* assignment gives every variable a value. A **solution** is a consistent, complete assignment.

Constraints may be *unary* (one variable, $X \neq 3$), *binary* (two variables, $X < Y$), or *global* (many variables, for example Alldiff). A binary CSP is drawn as a **constraint graph**: nodes are variables, edges are constraints.

Example: map colouring (Australia). Variables $X = \{WA, NT, Q, NSW, V, SA, T\}$; each domain is $\{R, G, B\}$; constraints require neighbours to differ: $SA \neq WA, SA \neq NT, SA \neq Q, SA \neq NSW, SA \neq V, WA \neq NT, NT \neq Q, Q \neq NSW, NSW \neq V$.

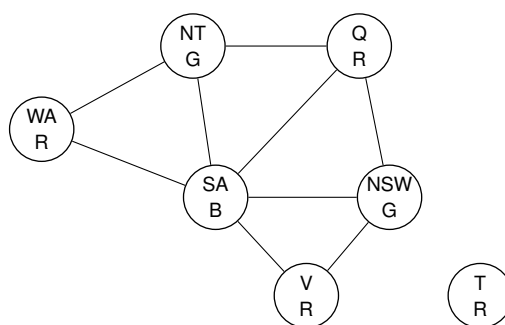


Figure 2: Constraint graph with one valid colouring. T has no neighbours, so any colour works.

Constraint propagation uses the constraints to shrink domains before or during search.

- **Node consistency:** every value in D_i satisfies the unary constraints on X_i . If $SA \neq G$, the domain of SA becomes $\{R, B\}$.
- **Arc consistency:** X_i is arc consistent with X_j if for every value in D_i there is some value in D_j that satisfies the constraint between them. The **AC3** algorithm keeps a queue of arcs, revises each, removes unsupported values, and requeues the neighbours of any domain that shrank; it runs in $O(cd^3)$ for c arcs and domain size d .
- **Path consistency:** a pair $\{X_i, X_j\}$ is path consistent with respect to X_m if every consistent assignment of the pair can be extended to X_m consistently with both. It tightens constraints between pairs and is stronger than arc consistency.

Example: $A, B, C \in \{1, 2, 3\}$ with $A < B$ and $B < C$. Revising arcs removes $A = 3$ (no larger B), $B = 1$ (no smaller A), $B = 3$ (no larger C), then $C = 1, 2$ and $A = 2$, leaving $A = 1, B = 2, C = 3$: propagation alone solves it.

Example: Three mutually different variables $X, Y, Z \in \{R, G\}$ are arc consistent, since each value has support in each neighbour. Path consistency detects the failure: for $\{X = R, Y = G\}$ no value of Z differs from both, so the pair is removed and the problem is shown to be unsolvable.

Backtracking search. Assign one variable at a time (depth first), check constraints after each assignment, and *backtrack* to the previous variable when no legal value remains. Assignments are commutative, so only one variable is chosen per level. It is improved by: *minimum remaining values* (choose the variable with the fewest legal values), the *degree heuristic* (most constraints on unassigned variables), *least constraining value* (try the value that rules out the fewest choices), and *forward checking* (after assigning X , delete inconsistent values from its neighbours and stop early if a domain becomes empty).

4 queens. One variable per row (its column), domains $\{1, 2, 3, 4\}$, constraints: no two in the same column or diagonal. Backtracking finds the solutions $(2, 4, 1, 3)$ and $(3, 1, 4, 2)$. The 8 queens problem is formulated the same way with 92 solutions.

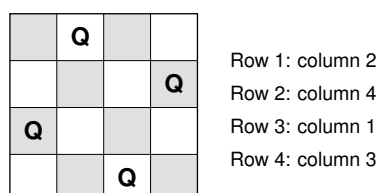


Figure 3: A solution of 4 queens: no two queens share a row, column or diagonal.

Cryptarithmic: $SEND + MORE = MONEY$. Each letter is a different digit; with column carries c_1, c_2, c_3 the constraints are $D + E = Y + 10c_1$, $N + R + c_1 = E + 10c_2$, $E + O + c_2 = N + 10c_3$, $S + M + c_3 = O + 10M$. Two four digit numbers sum to less than 20000, so $M = 1$. Then $S + 1 + c_3 = O + 10$ forces $O = 0$, with $S = 9$ when $c_3 = 0$. From $E + 0 + c_2 = N$ with $N \neq E$ we get $c_2 = 1$ and $N = E + 1$; then $N + R + c_1 = E + 10$ gives $R = 9 - c_1$, and since 9 is taken, $c_1 = 1$ and $R = 8$. The digits left for D, E, N, Y are $\{2, \dots, 7\}$ with $D + E = Y + 10$: $E = 5$, $N = 6$, $D = 7$, $Y = 2$. Check:

$$9567 + 1085 = 10652.$$

2.3 Search Algorithms, Strategies and Evaluation

Search finds a path through the state space by building a *search tree*. The **frontier** (open list) holds generated but unexpanded nodes; the **explored** (closed) set holds expanded ones. A strategy is simply the order in which frontier nodes are expanded: a FIFO queue gives BFS, a LIFO stack gives DFS, and a priority queue ordered by an evaluation function $f(n)$ gives best first search.

Measures used to evaluate a search strategy

Completeness: does it find a solution whenever one exists? **Optimality:** does it find the least cost solution? **Time complexity:** nodes generated. **Space complexity:** nodes held in memory. Here b is the branching factor, d the depth of the shallowest goal, m the maximum depth of the tree, and l the depth limit.

Strategies are grouped as **uninformed** (blind: BFS, DFS, depth limited, iterative deepening), **informed** or heuristic (best first, greedy, A*), **adversarial** (minimax, alpha beta) and **local search** (hill climbing, simulated annealing, genetic algorithm).

2.4 Uninformed Search

Uninformed strategies use only the problem definition; they cannot tell whether one non goal state is closer to the goal than another.

Breadth first search (BFS). Expand the root, then all nodes at depth 1, then depth 2, and so on, using a FIFO queue. It is complete (finite b) and optimal when every step costs the same. Nodes generated: $b + b^2 + \dots + b^d = O(b^d)$ in both time and space, and memory is the real limit. Uses: shortest path in unweighted graphs, friend suggestions “people you may know”.

Depth first search (DFS). Always expand the deepest node, using a LIFO stack; on a dead end it backtracks. It stores only one path plus the unexpanded siblings along it, so space is $O(bm)$, but time is $O(b^m)$. It is incomplete on infinite or cyclic spaces and not optimal, since it may return a deep solution while a shallow one exists. Uses: maze solving, file system traversal.

Depth limited search (DLS). DFS that treats nodes at depth l as having no successors. Time $O(b^l)$, space $O(bl)$. It stops infinite descent but is incomplete if $l < d$ and not optimal if $l > d$.

Iterative deepening search (IDS). Run DLS with $l = 0, 1, 2, \dots$ until the goal is found. It combines the completeness and optimality (unit step cost) of BFS with the $O(bd)$ memory of DFS. Shallow levels are regenerated, but this is cheap because most nodes are at the bottom: for $b = 10, d = 5$, IDS generates $50 + 400 + 3000 + 20000 + 100000 = 123,450$ nodes against 111,110 for BFS, only about 11% more.

Bidirectional search runs one search forward from the start and one backward from the goal and stops when they meet, giving $O(b^{d/2})$ time and space; it needs reversible actions and a known goal.



Figure 4: Order in which the same tree is expanded (numbers beside nodes).

Criterion	BFS	DFS	DLS	IDS	Bidirectional
Complete	Yes	No	No (if $l < d$)	Yes	Yes
Optimal	Yes (unit cost)	No	No	Yes (unit cost)	Yes (unit cost)
Time	$O(b^d)$	$O(b^m)$	$O(b^l)$	$O(b^d)$	$O(b^{d/2})$
Space	$O(b^d)$	$O(bm)$	$O(bl)$	$O(bd)$	$O(b^{d/2})$

2.5 Informed (Heuristic) Search

Informed search uses problem specific knowledge through a **heuristic function** $h(n)$, the estimated cost of the cheapest path from n to a goal, with $h(\text{goal}) = 0$. **Best first search** is a family: keep the frontier as a priority queue and always expand the node with the lowest evaluation $f(n)$; different choices of f give different algorithms.

- **Greedy best first search:** $f(n) = h(n)$. It always moves toward the node that looks closest to the goal and ignores the cost already paid. Neither complete (can loop) nor optimal; time and space $O(b^m)$ in the worst case, much better with a good heuristic.
- **A* search:** $f(n) = g(n) + h(n)$, where $g(n)$ is the actual cost from the start to n and $f(n)$ is the estimated cost of the cheapest solution through n .

Admissible and consistent heuristics

h is **admissible** if it never overestimates: $0 \leq h(n) \leq h^*(n)$, where $h^*(n)$ is the true cost to the goal. h is **consistent** (monotone) if $h(n) \leq c(n, a, n') + h(n')$ for every successor n' . Consistent implies admissible. A* is complete and *optimal* when h is admissible (tree search) or consistent (graph search), and no other optimal algorithm using the same heuristic expands fewer nodes. Its weakness is memory: it keeps all generated nodes, $O(b^d)$.

Heuristics for the 8 puzzle. h_1 = number of misplaced tiles; h_2 = sum of Manhattan distances of tiles from their goal squares. Both are admissible, and $h_2 \geq h_1$ so h_2 dominates. For start $\begin{smallmatrix} 2 & 8 & 3 \\ 1 & 6 & 4 \\ 7 & 5 & \end{smallmatrix}$ and goal $\begin{smallmatrix} 1 & 2 & 3 \\ 8 & 6 & 4 \\ 7 & 5 & \end{smallmatrix}$: $h_1 = 4$ (tiles 2, 8, 1, 6) and $h_2 = 1 + 2 + 1 + 1 = 5$.

Worked example: route from Arad to Bucharest. Heuristic h is the straight line distance to Bucharest; edge labels are road distances.

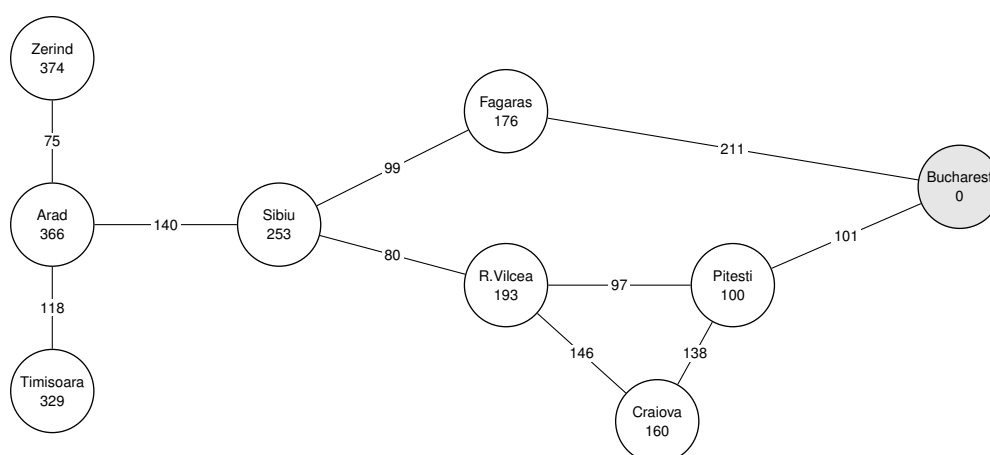


Figure 5: Road map fragment. Node label: city and h (straight line distance to Bucharest).

Greedy: Arad ($h = 366$) expands to Sibiu (253, the smallest), then Fagaras (176), then Bucharest: path cost $140 + 99 + 211 = 450$. **A*:** expansion order and $f = g + h$:

Step	Expand	g	h	f	Generated (with f)
1	Arad	0	366	366	Sibiu 393, Timisoara 447, Zerind 449
2	Sibiu	140	253	393	Fagaras 415, R.Vilcea 413, Arad 646
3	R.Vilcea	220	193	413	Pitesti 417, Craiova 526
4	Fagaras	239	176	415	Bucharest 450
5	Pitesti	317	100	417	Bucharest 418, Craiova 615 (worse than 526, ignored)
6	Bucharest	418	0	418	Goal reached

Bucharest first appears with $f = 450$ at step 4 but is not chosen, because Pitesti with $f = 417$ is cheaper; at step 5 a better route to Bucharest ($f = 418$) appears and is selected. A* returns Arad, Sibiu, R.Vilcea, Pitesti, Bucharest with cost **418**, better than greedy's 450.

Practice example (greedy fails, A* succeeds). Edge costs: S to A 1, S to B 4, A to C 2, A to D 5, B to D 1, C to G 6, D to G 2. Heuristic: $h(S) = 3, h(A) = 2, h(B) = 3, h(C) = 3, h(D) = 2, h(G) = 0$ (admissible and consistent). Greedy always picks the smallest h : S, A ($h = 2$), D ($h = 2$), G, cost $1 + 5 + 2 = 8$. A* keeps $f = g + h$:

Step	Expand	Frontier after expansion, as node ($g + h = f$)
1	S (3)	A ($1 + 2 = 3$), B ($4 + 3 = 7$)
2	A (3)	B (7), C ($3 + 3 = 6$), D ($6 + 2 = 8$)
3	C (6)	B (7), D (8), G ($9 + 0 = 9$)
4	B (7)	D ($5 + 2 = 7$, replaces 8), G (9)
5	D (7)	G ($7 + 0 = 7$, replaces 9)
6	G (7)	Goal: path S, B, D, G with cost 7 (optimal)

	Greedy best first	A*
Evaluation	$f(n) = h(n)$	$f(n) = g(n) + h(n)$
Uses path cost so far	No	Yes
Complete	No (may loop)	Yes
Optimal	No	Yes, if h is admissible
Speed and memory	Fast, but memory $O(b^m)$	Slower, memory $O(b^d)$

2.6 Adversarial Search

Adversarial search is used when two or more agents with conflicting goals act in the same environment, as in chess, checkers and tic tac toe. A two player, zero sum, turn taking game is defined by an initial state, the player to move, the legal actions, a terminal test, and a *utility* for terminal states (for example +1 win, 0 draw, -1 loss). One player is **MAX** (maximises utility), the other **MIN** (minimises it).

Minimax algorithm

Minimax computes the optimal move for MAX assuming MIN also plays optimally, by a depth first recursion over the game tree:

$$\text{MINIMAX}(s) = \begin{cases} \text{UTILITY}(s) & \text{if } s \text{ is terminal,} \\ \max_a \text{ MINIMAX}(\text{RESULT}(s, a)) & \text{if MAX moves in } s, \\ \min_a \text{ MINIMAX}(\text{RESULT}(s, a)) & \text{if MIN moves in } s. \end{cases}$$

Steps: (1) generate the tree to the terminal states; (2) assign utilities to the leaves; (3) back up values level by level, taking the maximum at MAX nodes and the minimum at MIN nodes; (4) at the root, MAX picks the move with the largest value. Minimax is complete on a finite tree and optimal against an optimal opponent; time $O(b^m)$, space $O(bm)$ with depth first generation. It is far too slow for games with large b , which motivates pruning.

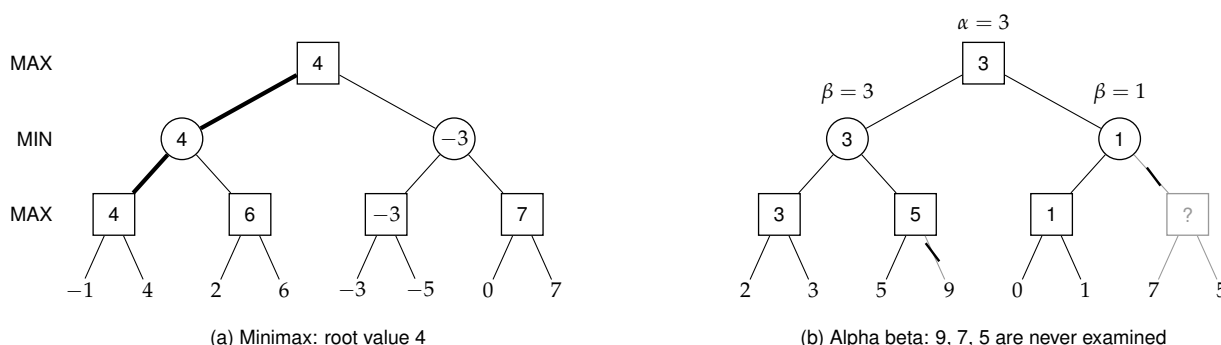


Figure 6: (a) Minimax values; the thick path is MAX's chosen line. (b) Alpha beta on another tree; grey branches are pruned, bars mark cut offs.

Alpha beta pruning

Alpha beta is minimax with two bounds carried down the tree: α , the best (highest) value MAX can already guarantee on the path, and β , the best (lowest) value MIN can already guarantee. Start with $\alpha = -\infty$, $\beta = +\infty$. MAX nodes raise α , MIN nodes lower β . **Prune the remaining children as soon as $\alpha \geq \beta$.** It returns exactly the same move as minimax but examines fewer nodes.

Trace of figure (b). Node D (MAX) sees 2 then 3, so $D = 3$; MIN node B gets $\beta = 3$. At E (MAX) the first leaf 5 gives $\alpha = 5 \geq \beta = 3$, so leaf 9 is pruned; $B = 3$ and the root gets $\alpha = 3$. At F leaves 0 and 1 give $F = 1$; MIN node C then has $\beta = 1 \leq \alpha = 3$, so the whole subtree G (leaves 7 and 5) is pruned. $C = 1$ and the root value is $\max(3, 1) = 3$.

Efficiency. Pruning depends on move ordering. With worst ordering nothing is pruned, $O(b^m)$; with best ordering only $O(b^{m/2})$ nodes are examined, so alpha beta can search about twice as deep in the same time. In practice, search is cut at a fixed depth and an *evaluation function* estimates the utility of non terminal states.

2.7 Local Search and Optimisation

Local search keeps only the current state and moves to neighbouring states; the *path is irrelevant*, only the final state matters. It uses very little memory and suits optimisation problems (timetables, circuit layout, routing) where the objective function is to be maximised or minimised.

Hill climbing. Repeatedly move to the neighbour with a better objective value and stop at a peak where no neighbour is better. Variants: *simple* (take the first better neighbour), *steepest ascent* (examine all neighbours, take the best), *stochastic* (pick a random better neighbour), and *random restart* (rerun from random starts).

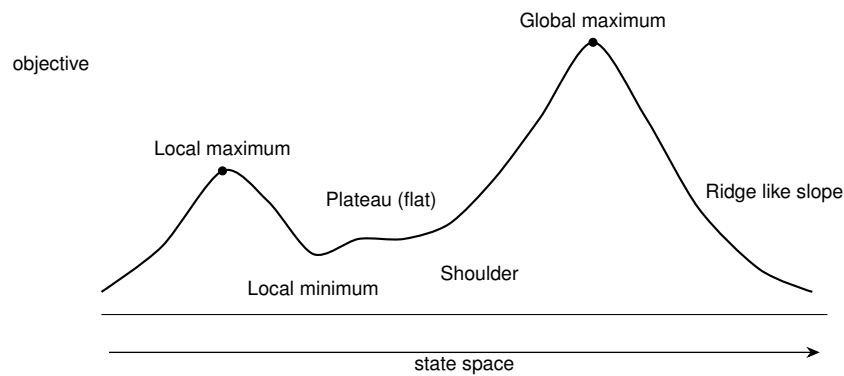


Figure 7: State space landscape. Hill climbing started on the left slope stops at the local maximum.

Hill climbing fails at **local maxima** (better than neighbours but not the best), **plateaus** (flat, no direction), **shoulders** (a plateau with an uphill edge) and **ridges** (a narrow high strip whose direction is not among the available moves). It is greedy, incomplete, and not optimal.

Simulated annealing

Inspired by annealing in metallurgy (heat, then cool slowly to reach a low energy crystal). Start from a random state with a high temperature T . At each step pick a random neighbour and let $\Delta E = E_{\text{new}} - E_{\text{current}}$ (change in *cost*). If $\Delta E < 0$ (better), accept it. Otherwise accept the worse state with probability

$$P = e^{-\Delta E/T}.$$

Lower the temperature by a cooling schedule, for example $T \leftarrow \alpha T$ with $0 < \alpha < 1$, and stop when T is near zero or steps run out.

Behaviour: at high T almost any move is accepted (exploration, like a random walk); as $T \rightarrow 0$ only improvements are accepted (like hill climbing). *Example:* For a worse move with $\Delta E = 4$: at $T = 10$, $P = e^{-0.4} = 0.67$; at $T = 1$, $P = e^{-4} = 0.018$. So early on it can walk down out of a local optimum, later it settles. With a sufficiently slow schedule it converges to the global optimum with probability one.

	Hill climbing	Simulated annealing
Worse moves	Never accepted	Accepted with probability $e^{-\Delta E/T}$
Local optima	Gets stuck	Can escape
Parameters	None (random restarts optional)	Initial T , cooling schedule, stopping rule
Memory	One state	One state
Speed	Fast	Slower; many iterations; sensitive to schedule

2.8 Evolutionary Optimisation: Genetic Algorithm

A **genetic algorithm** (GA) is a population based search inspired by natural selection. Terms: **population** (set of candidate solutions), **chromosome** (one solution, often a bit string), **gene** (one bit or parameter), **fitness function** (score; higher is better).

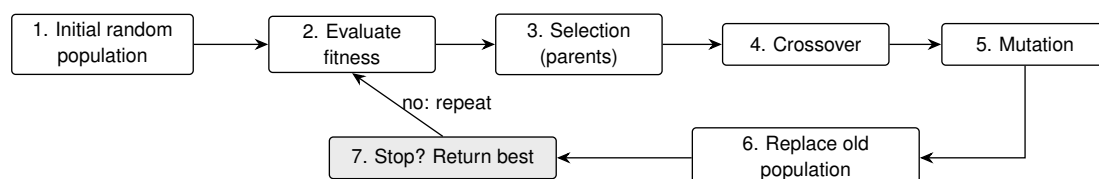


Figure 8: The genetic algorithm cycle.

Operators.

- **Selection:** fitter individuals are more likely to be parents. *Roulette wheel:* probability $p_i = f_i / \sum_j f_j$. Also tournament and rank selection.
- **Crossover:** combine two parents. *Single point:* choose a cut point and swap tails; *two point;* *uniform* (each gene from either parent). Applied with a crossover probability (often 0.6 to 0.9).
- **Mutation:** flip each bit with a small probability (for example 0.001 to 0.01) to keep diversity and escape local optima.
- **Replacement and termination:** the offspring replace all or part of the population (*elitism* keeps the best individuals); stop after a fixed number of generations, when fitness stops improving, or when a target is reached.

Worked example. Maximise $f(x) = x^2$ for $x \in \{0, \dots, 31\}$ using 5 bit strings and a population of 4.

No.	String	x	$f = x^2$	$p_i = f_i / \sum f$	Expected copies ($4p_i$)
1	01101	13	169	0.144	0.58
2	11000	24	576	0.492	1.97
3	01000	8	64	0.055	0.22
4	10011	19	361	0.309	1.23
		Sum	1170	Average	292.5

Selection: the mating pool becomes 01101, 11000, 11000, 10011 (string 3 is dropped, string 2 is duplicated). **Crossover:** pair 01101 with 11000 at cut point 4 and 11000 with 10011 at cut point 2:

$$0110|1 \times 1100|0 \rightarrow 01100, 11001; \quad 11|000 \times 10|011 \rightarrow 11011, 10000.$$

New population: 01100 (12, $f = 144$), 11001 (25, 625), 11011 (27, 729), 10000 (16, 256). The sum is 1754 and the average **438.5**, up from 292.5, and the best fitness rose from 576 to 729 after one generation; mutation is omitted here for clarity.

Strengths: works without gradients, handles discrete and noisy problems, explores many regions at once. **Limits:** no guarantee of the optimum, sensitive to encoding and parameters, needs many fitness evaluations, may converge early to a local optimum.

Chapter 3 Knowledge Representation and Probabilistic Reasoning

3.1 Knowledge Based Agent

Knowledge based agent

A knowledge based agent keeps an internal **knowledge base (KB)**, a set of sentences in a formal language, and uses an **inference engine** to derive new sentences from it and decide what to do. Knowledge is more than data: it is information that has been interpreted, organised and made ready to act on. Types: *declarative* (facts, knowing what), *procedural* (rules, knowing how), *heuristic* (expert rules of thumb), *meta knowledge* (knowledge about knowledge), *structural* (relations between concepts).

Each time the agent program is called it does three things: it **TELLs** the KB what it perceives; it **ASKs** the KB what action to take (reasoning happens here); then it TELLs the KB which action it chose and executes it.

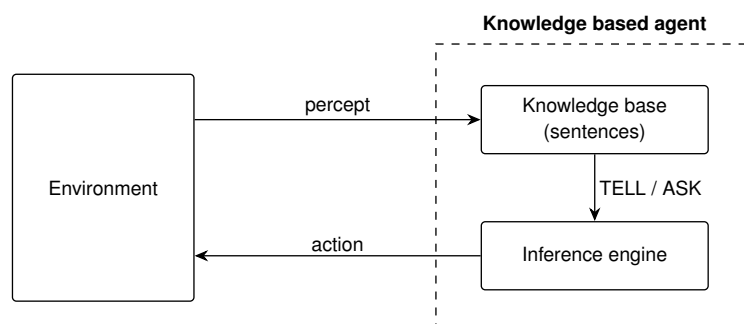


Figure 9: TELL, ASK and act cycle of a knowledge based agent.

Example: the Wumpus world. A 4×4 cave; the agent starts at $[1, 1]$ and must find the gold and climb out. Hazards: a *Wumpus* (kills the agent) and bottomless *pits*. **Percepts:** Stench (in a room next to the Wumpus), Breeze (next to a pit), Glitter (gold here), Bump (walked into a wall), Scream (Wumpus killed). **Actions:** forward, turn left, turn right, grab, shoot (one arrow), climb. **Performance:** +1000 for leaving with the gold, -1000 for death, -1 per action, -10 for using the arrow.

4				
3	Wumpus			
2	S OK	OK (safe)		
1	A OK	B OK	Pit	
	1	2	3	4

A = agent, B = breeze, S = stench
Grey squares are deduced, not seen.

1. $[1, 1]$: no percept, so $[1, 2]$ and $[2, 1]$ are safe.
2. At $[2, 1]$: breeze, so a pit is in $[2, 2]$ or $[3, 1]$.
3. At $[1, 2]$: stench, so the Wumpus is in $[1, 3]$ or $[2, 2]$.
4. $[2, 1]$ had no stench, so $[2, 2]$ has no Wumpus; hence the Wumpus is at $[1, 3]$.
5. $[1, 2]$ had no breeze, so $[2, 2]$ has no pit; hence the pit is at $[3, 1]$ and $[2, 2]$ is safe to enter.

Figure 10: Logical inference in the Wumpus world (column, row).

Knowledge engineering is the process of building a KB, done by a knowledge engineer working with a domain expert. Five steps: (1) identify the task; (2) assemble the relevant knowledge from the expert; (3) decide on a vocabulary of predicates, functions and constants; (4) encode the general knowledge as rules or logic; (5) debug, test and refine the KB until it answers correctly.

3.2 Knowledge Representation Techniques and Issues

A good representation has four properties: *representational adequacy* (can express all the needed knowledge), *inferential adequacy* (can derive new knowledge from it), *inferential efficiency* (can steer inference in productive directions), *acquisitional efficiency* (new knowledge can be added easily, ideally automatically).

Approach	Idea	Examples
Logical	Facts as sentences of a formal logic with syntax and semantics.	Propositional logic $P \rightarrow Q$; predicate logic $\forall x \text{ Human}(x) \rightarrow \text{Mortal}(x)$.
Procedural	Knowledge as procedures: (condition, action) pairs.	Production rules: IF at bus stop AND bus arrives THEN board.
Network	Graph: nodes are objects or events, arcs are relations.	Semantic networks, conceptual graphs.
Structured	Nodes with internal structure: named slots holding values.	Frames, scripts, objects.

Issues in representation.

- **Important attributes:** choose the attributes that matter (many are slow, few are useless); attributes such as *instance* and *isa* support inheritance.
- **Relationships and inverses:** facts are linked, and inverse relations (owns versus owned by) trade memory against processing speed.
- **Granularity:** how detailed the knowledge should be; too coarse gives errors, too fine gives slow, confused reasoning.
- **Representing sets:** describe classes by their defining properties (intension) rather than listing every member (extension).
- **Finding the right structure:** selecting the right frame or schema for a new situation.

Path planning is a representation problem too: the world is modelled as a graph or grid of states, and a route is found with a search algorithm such as A^* , using the geometric distance as heuristic.

3.3 Propositional and Predicate Logic

Propositional logic (PL) works with *propositions*, declarative statements that are true or false (questions and commands are not). Atomic propositions (P, Q, R) are joined by connectives to make compound ones. Precedence, high to low: $\neg, \wedge, \vee, \rightarrow, \leftrightarrow$.

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \rightarrow Q$	$P \leftrightarrow Q$
F	F	T	F	F	T	T
F	T	T	F	T	T	F
T	F	F	F	T	F	F
T	T	F	T	T	T	T

A formula is a **tautology** (valid) if true under every interpretation (for example $P \vee \neg P$), a **contradiction** (unsatisfiable) if always false, and a **contingency** (satisfiable) otherwise. $KB \models \alpha$ (entailment) means α is true in every model in which the KB is true; equivalently $KB \wedge \neg \alpha$ is unsatisfiable, which is the basis of refutation proofs. A *well formed formula* (WFF) is a syntactically legal expression; $P \wedge \vee Q$ and $(P \wedge Q \vee R)$ are not.

Key equivalences. De Morgan: $\neg(P \wedge Q) \equiv \neg P \vee \neg Q$, $\neg(P \vee Q) \equiv \neg P \wedge \neg Q$. Implication: $P \rightarrow Q \equiv \neg P \vee Q$. Contrapositive: $P \rightarrow Q \equiv \neg Q \rightarrow \neg P$. Biconditional: $P \leftrightarrow Q \equiv (P \rightarrow Q) \wedge (Q \rightarrow P)$. Distribution: $P \vee (Q \wedge R) \equiv (P \vee Q) \wedge (P \vee R)$. Double negation: $\neg \neg P \equiv P$.

Limits of PL: it cannot look inside a sentence, has no quantifiers, and cannot say “all kings are men” in general.

Rules of inference

Rule	Form	Example
Modus ponens	$P, P \rightarrow Q \therefore Q$	It is raining; if it rains the street is wet; so the street is wet.
Modus tollens	$\neg Q, P \rightarrow Q \therefore \neg P$	The street is not wet; so it did not rain.
Hypothetical syllogism	$P \rightarrow Q, Q \rightarrow R \therefore P \rightarrow R$	Rain implies tea; tea implies reading; so rain implies reading.
Disjunctive syllogism	$P \vee Q, \neg P \therefore Q$	Tea or coffee; no tea; so coffee.
Addition	$P \therefore P \vee Q$	It is raining, so it is raining or cold.
Simplification	$P \wedge Q \therefore P$	Hot and humid, so hot.
Conjunction	$P, Q \therefore P \wedge Q$	Combine two known facts.
Resolution	$P \vee Q, \neg P \vee R \therefore Q \vee R$	Eliminates a complementary pair of literals.

Example: Premises: if I play hockey I am sore ($p \rightarrow q$); I use the whirlpool if I am sore ($q \rightarrow r$); I did not use the whirlpool ($\neg r$). Hypothetical syllogism on the first two gives $p \rightarrow r$; modus tollens with $\neg r$ gives $\neg p$: I did not play hockey.

Example: If you email me, I finish the program ($p \rightarrow q$); if you do not email, I sleep early ($\neg p \rightarrow r$); if I sleep early, I wake refreshed ($r \rightarrow s$). Prove: if I do not finish, I wake refreshed. (1) $\neg q \rightarrow \neg p$ (contrapositive of $p \rightarrow q$); (2) $\neg q \rightarrow r$ (hypothetical syllogism, 1 and $\neg p \rightarrow r$); (3) $\neg q \rightarrow s$ (hypothetical syllogism, 2 and $r \rightarrow s$).

Predicate (first order) logic

FOL adds *objects* (constants, variables), *predicates* (properties and relations, returning true or false), *functions* and *quantifiers*. $\forall x P(x)$: for all x . $\exists x P(x)$: there is at least one x . Negation: $\neg \forall x P \equiv \exists x \neg P$ and $\neg \exists x P \equiv \forall x \neg P$. Typical pairing: $\forall$ goes with $\rightarrow$, $\exists$ goes with $\wedge$.

English	FOL
All birds fly.	$\forall x (\text{Bird}(x) \rightarrow \text{Fly}(x))$
Some boys play cricket.	$\exists x (\text{Boy}(x) \wedge \text{Plays}(x, \text{cricket}))$
No purple mushroom is poisonous.	$\forall x (\text{Purple}(x) \wedge \text{Mushroom}(x) \rightarrow \neg \text{Poisonous}(x))$
Not all students like both Maths and Science.	$\neg \forall x (\text{Student}(x) \rightarrow \text{Likes}(x, M) \wedge \text{Likes}(x, S))$
Every student has a professor who teaches them.	$\forall x (\text{Student}(x) \rightarrow \exists y (\text{Prof}(y) \wedge \text{Teaches}(y, x)))$

A **Horn clause** is a disjunction of literals with at most one positive literal, such as $\neg Q \vee \neg R \vee P$, which reads $(Q \wedge R) \rightarrow P$. **Universal instantiation:** from $\forall x P(x)$ infer $P(c)$ for any constant c . **Existential instantiation:** from $\exists x P(x)$ infer $P(k)$ for a *new* constant k . **Unification** finds a substitution making two literals identical: $\text{Loves}(x, \text{Mother}(y))$ and $\text{Loves}(\text{John}, z)$ unify with $\theta = \{x/\text{John}, z/\text{Mother}(y)\}$.

Conversion to clause (conjunctive normal) form

CNF is a conjunction of clauses, each a disjunction of literals. Steps: (1) eliminate $\leftrightarrow$ and $\rightarrow$; (2) move $\neg$ inward (De Morgan, quantifier negation); (3) standardise variable names apart; (4) **Skolemise:** replace each $\exists$ variable by a constant, or by a function of the enclosing $\forall$ variables; (5) drop the $\forall$ quantifiers; (6) distribute $\vee$ over $\wedge$.

Example: $\neg(P \wedge Q) \vee R \equiv \neg P \vee \neg Q \vee R$ (one clause). $\neg(P \vee Q) \wedge (R \rightarrow S) \equiv \neg P \wedge \neg Q \wedge (\neg R \vee S)$ (three clauses).

Example: “Everyone who loves all animals is loved by someone.”

$$\begin{aligned} & \forall x [\forall y \text{ Animal}(y) \rightarrow \text{Loves}(x, y)] \rightarrow [\exists y \text{ Loves}(y, x)] \\ & \forall x [\exists y \text{ Animal}(y) \wedge \neg \text{Loves}(x, y)] \vee [\exists z \text{ Loves}(z, x)] \quad (\text{steps 1 to 3}) \\ & \forall x [\text{Animal}(F(x)) \wedge \neg \text{Loves}(x, F(x))] \vee \text{Loves}(G(x), x) \quad (\text{Skolem functions } F, G) \\ & [\text{Animal}(F(x)) \vee \text{Loves}(G(x), x)] \wedge [\neg \text{Loves}(x, F(x)) \vee \text{Loves}(G(x), x)] \end{aligned}$$

Resolution refutation

Resolution (Robinson, 1965) is one inference rule that is sound and refutation complete for clauses. To prove a goal G from a KB: (1) write the KB in FOL; (2) convert to clauses; (3) *negate* the goal and add it; (4) repeatedly resolve pairs of clauses containing complementary (unified) literals; (5) if the *empty clause* is derived, the assumption $\neg G$ is contradictory, so G is proved.

Example: KB: P ; $(P \wedge Q) \rightarrow R$; $(S \vee T) \rightarrow Q$; T . Prove R . Clauses: (1) P ; (2) $\neg P \vee \neg Q \vee R$; (3) $\neg S \vee Q$; (4) $\neg T \vee Q$; (5) T ; negated goal (6) $\neg R$.

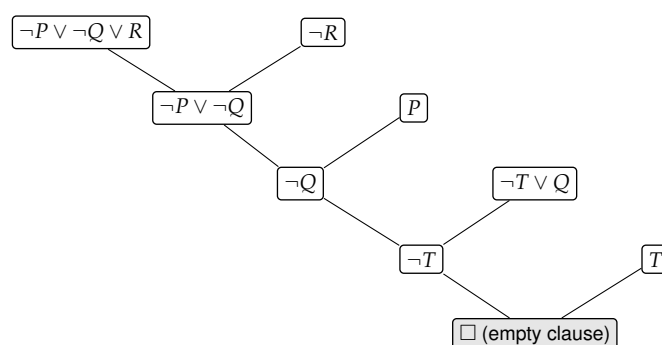


Figure 11: Resolution refutation. The contradiction shows R follows from the KB.

Example: (FOL) Premises: $\text{Dog}(\text{Fido})$; $\forall x \text{ Dog}(x) \rightarrow \text{Animal}(x)$; $\forall x \text{ Animal}(x) \rightarrow \text{Dies}(x)$. Prove $\text{Dies}(\text{Fido})$. Clauses: $\text{Dog}(\text{Fido})$; $\neg \text{Dog}(x) \vee \text{Animal}(x)$; $\neg \text{Animal}(x) \vee \text{Dies}(x)$; negated goal $\neg \text{Dies}(\text{Fido})$. Resolving the goal with the third gives $\neg \text{Animal}(\text{Fido})$ (with x/Fido), then with the second gives $\neg \text{Dog}(\text{Fido})$, which clashes with the first: the empty clause.

Forward and backward chaining

Both work on Horn clause KBs (rules $p_1 \wedge \dots \wedge p_k \Rightarrow q$ and facts) and are sound and complete for them, in time linear in the size of the KB.

- **Forward chaining** (data driven): start from the known facts; fire every rule whose premises are all known and add its conclusion; repeat until the query is added or nothing new appears.
- **Backward chaining** (goal driven): start from the query; find rules that conclude it, and make their premises new sub goals; recurse until every sub goal is a known fact. It is essentially AND OR graph search.

Example: Facts: A, B . Rules: (1) $P \Rightarrow Q$; (2) $L \wedge M \Rightarrow P$; (3) $B \wedge L \Rightarrow M$; (4) $A \wedge P \Rightarrow L$; (5) $A \wedge B \Rightarrow L$. Query: Q .

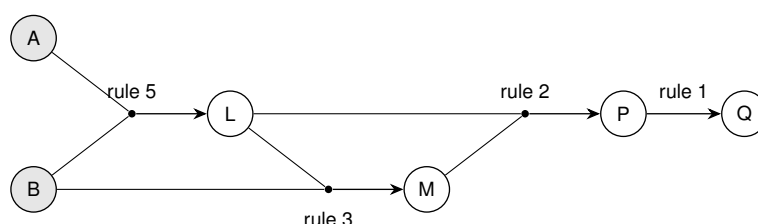


Figure 12: AND graph of the rules (black dots join the premises of one rule; shaded nodes are known facts).

Forward: known $\{A, B\}$; rule 5 fires, add L ; rule 3 (B, L) adds M ; rule 2 (L, M) adds P ; rule 1 adds Q . Success. (Rule 4 also fires but L is already known.) **Backward:** Q needs P ; P needs L and M ; L needs A, B (both facts, so L holds); M needs B, L (both hold); hence P , hence Q .

	Forward chaining	Backward chaining
Direction	Facts to conclusion	Query back to facts
Driven by	Data	Goal
Effort	May derive many irrelevant facts	Focused on the query
Typical use	Monitoring, production systems, expert system control	Diagnosis, question answering, Prolog

3.4 Semantic Networks and Frames

Semantic network

A semantic network is a labelled directed graph: nodes are objects, classes or values, and arcs are relations such as *is a* (class membership or subclass), *has part*, *owned by*, *colour*. **Inheritance:** a node inherits the properties of every class above it along *is a* links, so facts are stored once.

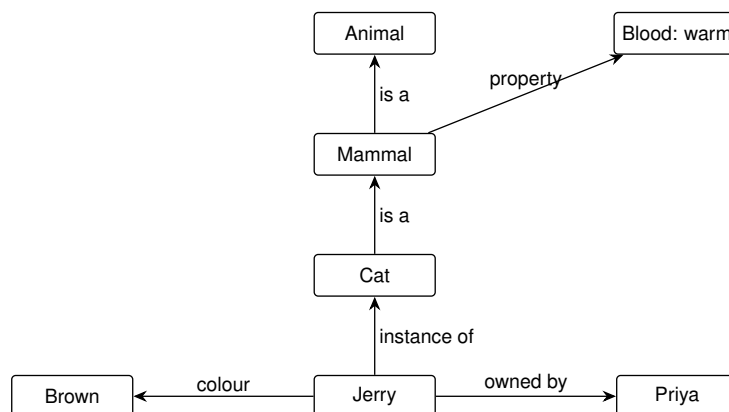


Figure 13: Semantic network for “Jerry is a cat, a mammal, owned by Priya, brown; all mammals are animals”. Jerry inherits “warm blooded” and “animal”.

Frame

A frame is a structured record describing a stereotyped object or situation: a set of named **slots** holding values. Slots may have *default values*, *restrictions* (type, range) and *attached procedures* (if needed, if added). Frames are organised in a hierarchy and child frames inherit slots from parents.

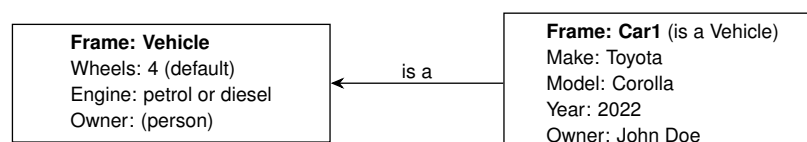


Figure 14: Frames with inheritance: Car1 inherits Wheels = 4 from Vehicle.

Semantic network versus frame. A network is good for showing relations and inheritance across many objects graphically; a frame packs all the attributes of one object together and supports defaults and procedures. Both are easy to read, but neither has a precise logical semantics, and exceptions and negation are awkward.

3.5 Probabilistic Reasoning and Bayes Theorem

Real agents face uncertainty from partial observability and noisy sensors, so logic is replaced or extended by probability. **Prior** $P(A)$: belief before evidence. **Conditional** $P(A | B)$: belief in A given B . **Product rule**: $P(A \wedge B) = P(A | B)P(B)$. **Total probability (marginalisation)**: $P(B) = \sum_i P(B | A_i)P(A_i)$.

Bayes theorem

$$P(A | B) = \frac{P(B | A) P(A)}{P(B)}, \quad P(B) = P(B | A)P(A) + P(B | \neg A)P(\neg A).$$

$P(A)$ is the prior, $P(B | A)$ the likelihood, $P(B)$ the evidence (normaliser), and $P(A | B)$ the posterior: the updated belief in hypothesis A after seeing evidence B .

Example: In a county 51% of adults are male and 49% female; 9.5% of males and 1.7% of females smoke cigars. A randomly chosen adult smokes a cigar. Prior $P(M) = 0.51$. Evidence: $P(C) = 0.095(0.51) + 0.017(0.49) = 0.04845 + 0.00833 = 0.05678$. Posterior:

$$P(M | C) = \frac{0.095 \times 0.51}{0.05678} = 0.853.$$

Example: A disease affects 1% of people; the test is positive for 90% of the sick and, wrongly, for 9.6% of the healthy. $P(\text{sick} | +) = \frac{0.9(0.01)}{0.9(0.01) + 0.096(0.99)} = \frac{0.009}{0.10404} = 0.0865$: even after a positive test the chance of disease is only about 8.7%, because the disease is rare.

Bayesian (belief) networks

Bayesian network

A Bayesian network is a directed acyclic graph in which each node is a random variable and an arrow $X \rightarrow Y$ means X directly influences Y . Each node stores a **conditional probability table (CPT)** $P(X_i | \text{Parents}(X_i))$. Each variable is conditionally independent of its non descendants given its parents, so the full joint distribution factorises:

$$P(x_1, \dots, x_n) = \prod_{i=1}^n P(x_i | \text{parents}(X_i)).$$

Burglar alarm example. An alarm rings on a burglary B or an earthquake E ; neighbours John (J) and Mary (M) call when they hear it.

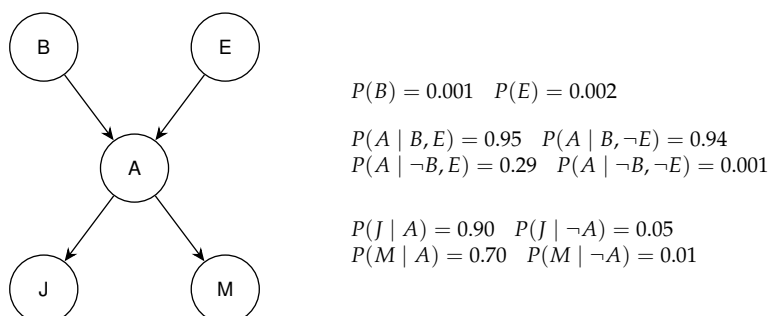


Figure 15: Alarm network with its CPT entries.

Joint probability: both call, alarm rings, no burglary, no earthquake:

$$\begin{aligned}
 P(J, M, A, \neg B, \neg E) &= P(J | A) P(M | A) P(A | \neg B, \neg E) P(\neg B) P(\neg E) \\
 &= 0.90 \times 0.70 \times 0.001 \times 0.999 \times 0.998 \approx 0.00063.
 \end{aligned}$$

Inference by enumeration: $P(B | J, M) = \frac{P(B, J, M)}{P(J, M)}$, where each term is a sum of joint products over the hidden variables E and A . Here $P(B, J, M) = 0.000592$ and $P(J, M) = 0.002084$, so $P(B | J, M) \approx 0.284$: even when both neighbours call, a burglary is far from certain.

3.6 Fuzzy Logic

Fuzzy logic

Fuzzy logic is many valued logic in which the truth of a statement is a degree between 0 and 1, modelling vague terms such as “warm” or “tall”. A **fuzzy set** A on universe X is described by a **membership function** $\mu_A : X \rightarrow [0, 1]$.

	Crisp (classical) set	Fuzzy set
Membership	Exactly 0 or 1 (in or out)	Any value in $[0, 1]$ (partial)
Logic	Two valued, Boolean	Infinite valued
Example	“She is 18 years old”	“She is about 18 years old”
Used in	Digital design	Fuzzy controllers

Membership functions translate a crisp number (35°C) into a degree of membership (0.8 “hot”). Common shapes:

$$\text{Triangular } (a, b, c) : \mu(x) = \max\left(\min\left(\frac{x-a}{b-a}, \frac{c-x}{c-b}\right), 0\right); \quad \text{Gaussian: } \mu(x) = e^{-\frac{(x-c)^2}{2\sigma^2}}.$$

The *trapezoidal* (a, b, c, d) rises on $[a, b]$, equals 1 on $[b, c]$ and falls on $[c, d]$.

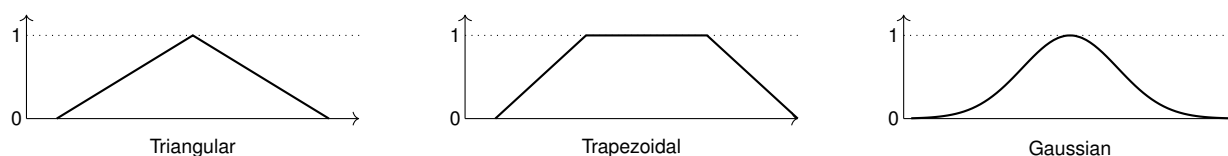


Figure 16: Common membership function shapes; the vertical axis is the degree of membership μ .

Operations on fuzzy sets: union $\mu_{A \cup B} = \max(\mu_A, \mu_B)$, intersection $\mu_{A \cap B} = \min(\mu_A, \mu_B)$, complement $\mu_{\bar{A}} = 1 - \mu_A$. A *linguistic variable* (temperature) takes fuzzy values (cold, warm, hot) defined by membership functions.

Fuzzy inference system (FIS)

A FIS maps crisp inputs to a crisp output using fuzzy IF THEN rules. Components: **fuzzifier** (crisp input to membership degrees), **rule base** (expert IF THEN rules), **inference engine** (evaluates rules, AND as min, combines fired rules, OR as max) and **defuzzifier** (fuzzy output to crisp value).

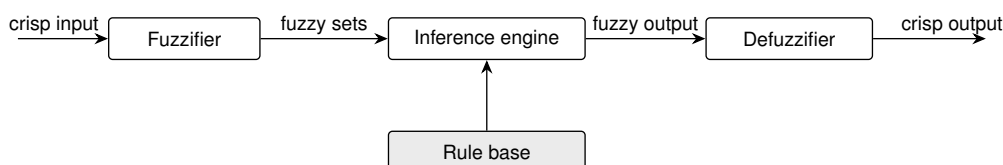


Figure 17: Architecture of a fuzzy inference system.

Worked example (fan speed). Inputs: temperature = 30°C , humidity = 65%.

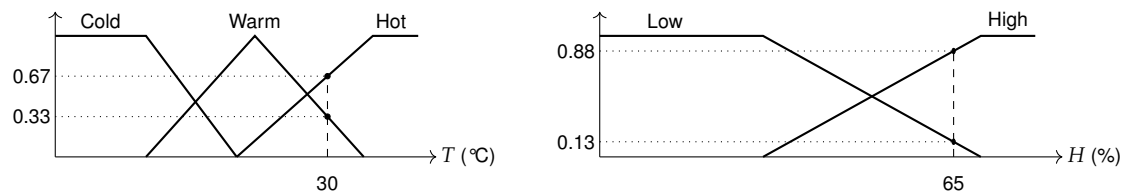


Figure 18: Fuzzification: $\mu_{\text{Warm}}(30) = 0.33$, $\mu_{\text{Hot}}(30) = 0.67$; $\mu_{\text{Low}}(65) = 0.125$, $\mu_{\text{High}}(65) = 0.875$.

Rules and firing strength (AND is min): R1 Hot AND High $\Rightarrow$ Fast: $\min(0.67, 0.875) = 0.67$. R2 Warm AND Low $\Rightarrow$ Slow: $\min(0.33, 0.125) = 0.125$. R3 Hot AND Low $\Rightarrow$ Medium: $\min(0.67, 0.125) = 0.125$. R4 Warm AND High $\Rightarrow$ Medium: $\min(0.33, 0.875) = 0.33$. Aggregate (OR is max): Slow 0.125, Medium $\max(0.125, 0.33) = 0.33$, Fast 0.67.

Defuzzification. The *centroid* method gives $z^* = \frac{\int z \mu(z) dz}{\int \mu(z) dz}$. Taking each output set at its centre (Slow 20, Medium 50, Fast 90), it becomes a weighted average:

$$z^* = \frac{0.125(20) + 0.33(50) + 0.67(90)}{0.125 + 0.33 + 0.67} = \frac{79.2}{1.125} \approx 70\% \text{ fan speed.}$$

Other methods: mean of maxima, and largest of maxima. **Applications:** washing machines, air conditioners, anti lock brakes, camera focus, medical diagnosis, industrial control.

Chapter 4 Machine Learning Fundamentals

4.1 Foundations and Four Pillars of Machine Learning

Machine learning

Machine learning (ML) lets a computer improve at a task from data instead of being explicitly programmed for every case (Samuel). Formally (Mitchell): a program learns from **experience** E with respect to a **task** T and **performance measure** P if its performance on T , measured by P , improves with E . *Example:* spam filtering; T classify emails, E thousands of labelled emails, P accuracy.

Vocabulary. *Examples:* items used for learning and testing. *Features:* the attributes describing an example (as a vector $\mathbf{x}$). *Labels:* the values or categories to predict (y). *Training sample:* labelled examples used to fit the model. *Test sample:* unseen examples used to measure performance. *Loss function:* measures the gap between predicted and true label.

Following Deisenroth, Faisal and Ong, ML rests on **six mathematical foundations** and revolves around **four pillars**, the fundamental problems that almost every ML method solves.

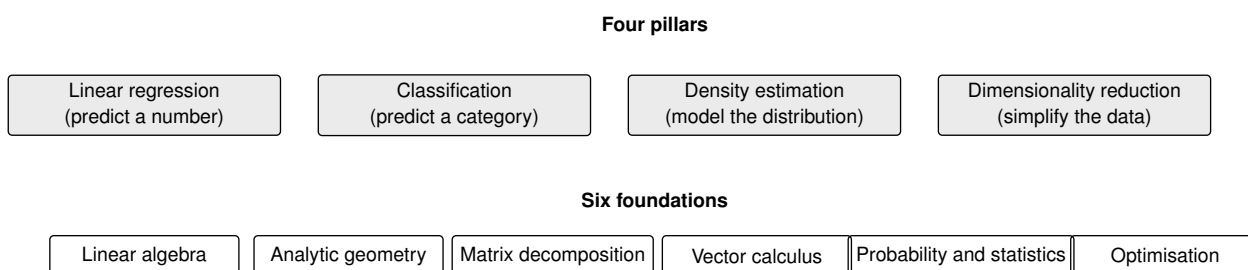


Figure 19: Foundations and pillars of machine learning.

Pillars. *Regression:* continuous output (house price, marks). *Classification:* discrete output (spam or not, disease or healthy). *Density estimation:* learn how data is distributed, to detect rare or anomalous points. *Dimensionality reduction:* keep the information in fewer features (PCA, t-SNE). **Foundations.** Data are vectors and matrices (linear algebra); similarity is distance or angle (analytic geometry); redundancy is removed by matrix decomposition (PCA); uncertainty is handled by probability; gradients come from vector calculus; and learning is optimisation of a loss.

4.2 Review of Mathematics for Machine Learning

Vectors and vector space. A vector is an ordered list of numbers, $\mathbf{x} = (x_1, \dots, x_n)$. A *vector space* is a set closed under vector addition and scalar multiplication; $\mathbb{R}^n$ is the standard example, and each data point is a vector in it. *Addition:* $(2, 3) + (5, 4) = (7, 7)$. *Scalar multiple:* $3(2, 5) = (6, 15)$.

Linear combination. $\mathbf{y} = c_1 \mathbf{v}_1 + \dots + c_k \mathbf{v}_k$. Almost every ML model is one: linear regression $y = w_1 x_1 + w_2 x_2 + b$. *Example:* $2\begin{pmatrix} 1 \\ 2 \end{pmatrix} + 3\begin{pmatrix} 3 \\ 1 \end{pmatrix} = \begin{pmatrix} 11 \\ 7 \end{pmatrix}$. The set of all linear combinations of given vectors is their **span**.

Linear independence. Vectors are independent if $c_1 \mathbf{v}_1 + \dots + c_k \mathbf{v}_k = \mathbf{0}$ only when all $c_i = 0$; otherwise one can be written from the others (redundant). $(1, 2)$ and $(2, 4)$ are dependent since $(2, 4) = 2(1, 2)$; $(1, 0)$ and $(0, 1)$ are independent. In ML, dependent features (height in cm and in m) add nothing. *Example:* For four houses the feature columns are Bedrooms $F_1 = (1, 2, 3, 4)$, Rooms $F_2 = (2, 6, 5, 4)$ and Master bedrooms $F_3 = (2, 4, 6, 8) = 2F_1$: F_3 is redundant and can be dropped, keeping F_1 and F_2 .

Basis and dimension. A *basis* is a set of independent vectors whose span is the whole space; the number of basis vectors is the *dimension* ($\mathbb{R}^n$ has dimension n). Coordinates of a vector depend on the basis. *Example:* Express $\mathbf{x} = (8, 2)$ in the basis $\mathbf{b}_1 = (1, 1)$, $\mathbf{b}_2 = (1, -1)$: solve $a + c = 8$, $a - c = 2$, giving $a = 5$, $c = 3$; check $5(1, 1) + 3(1, -1) = (8, 2)$. Bases underlie PCA (new axes) and feature extraction.

Geometry. Norm $\|\mathbf{x}\| = \sqrt{\sum x_i^2}$; dot product $\mathbf{x} \cdot \mathbf{y} = \sum x_i y_i = \|\mathbf{x}\| \|\mathbf{y}\| \cos \theta$. Similarity measures: Euclidean distance $\sqrt{\sum (x_i - y_i)^2}$, Manhattan distance $\sum |x_i - y_i|$, cosine similarity $\frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|}$. A matrix W acts as a linear transformation $\mathbf{y} = W\mathbf{x} + \mathbf{b}$, the building block of neural network layers.

4.3 Continuous Optimisation

An optimisation problem has an **objective (loss) function** $f(\mathbf{x})$ to minimise, **decision variables** $\mathbf{x}$ (weights), and optional **constraints**. Learning means finding the parameters with least loss.

Unconstrained optimisation. The *gradient* $\nabla f = (\frac{\partial f}{\partial x_1}, \dots, \frac{\partial f}{\partial x_n})$ points in the direction of steepest increase; a minimum has $\nabla f = \mathbf{0}$ (a stationary point).

Gradient descent

Move against the gradient with a step size η (learning rate): $\mathbf{x}_{t+1} = \mathbf{x}_t - \eta \nabla f(\mathbf{x}_t)$. Too small an η is slow; too large overshoots and can diverge.

Example: $f(x) = x^2$, $f'(x) = 2x$, start $x_0 = 4$, $\eta = 0.1$: $x_1 = 4 - 0.1(8) = 3.2$, $x_2 = 3.2 - 0.1(6.4) = 2.56$, $x_3 = 2.048, \dots$ each step multiplies x by 0.8 and it converges to 0. With $\eta = 1.1$ the factor is -1.2 , so the iterates oscillate and diverge.

Stochastic and mini batch. For $L(\theta) = \frac{1}{N} \sum_i L_i(\theta)$ full gradient descent costs $O(N)$ per step. *Stochastic gradient descent (SGD)* uses one random example, $\theta \leftarrow \theta - \eta \nabla L_i(\theta)$; *mini batch* averages over B examples. The gradient is noisy but cheap and the noise helps escape shallow minima. *Momentum* smooths it: $v \leftarrow \beta v + \nabla L$, $\theta \leftarrow \theta - \eta v$.

Beyond gradient descent: Jacobian and Hessian. For a vector valued $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^m$ the **Jacobian** is the $m \times n$ matrix of first partial derivatives, $J_{ij} = \partial f_i / \partial x_j$ (row i is the gradient of f_i); it generalises the derivative, and the chain rule becomes $J_{g \circ f} = J_g J_f$, which is what backpropagation multiplies. *Example:*

$$\mathbf{f}(x, y) = (x^2 y, 5x + \sin y): J = \begin{pmatrix} 2xy & x^2 \\ 5 & \cos y \end{pmatrix}, \text{ at } (1, 0): \begin{pmatrix} 0 & 1 \\ 5 & 1 \end{pmatrix}.$$

The **Hessian** H is the $n \times n$ matrix of second derivatives, $H_{ij} = \partial^2 f / \partial x_i \partial x_j$; it measures curvature. At a stationary point: H positive definite (all eigenvalues > 0) gives a *minimum*; negative definite gives a *maximum*; mixed signs give a *saddle point*. For 2×2 : $f_{xx} > 0$ and $\det H > 0$ means minimum. Newton's method uses it: $\mathbf{x}_{t+1} = \mathbf{x}_t - H^{-1} \nabla f$.

Example: $f(x, y) = x^2 + xy + y^2 - 3x$. $\nabla f = (2x + y - 3, x + 2y) = \mathbf{0}$ gives $y = -x/2$, $x = 2$, $y = -1$. $H = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$, $f_{xx} = 2 > 0$, $\det H = 3 > 0$: a minimum, with $f(2, -1) = -3$; H is constant and positive definite, so f is convex and this is the *global* minimum.

Example: $g(x) = x^4 - 4x^2$: $g' = 4x^3 - 8x = 0$ gives $x = 0, \pm\sqrt{2}$; $g'' = 12x^2 - 8$ is -8 at 0 (local maximum) and 16 at $\pm\sqrt{2}$ (minima, $g = -4$).

Constrained optimisation. General form: minimise $f(\mathbf{x})$ subject to $g_i(\mathbf{x}) \leq 0$ and $h_j(\mathbf{x}) = 0$.

Equality constraints: Lagrange multipliers. Form $\mathcal{L}(\mathbf{x}, \lambda) = f(\mathbf{x}) + \lambda h(\mathbf{x})$ and solve $\nabla \mathcal{L} = \mathbf{0}$. At the optimum ∇f is parallel to ∇h (level set of f tangent to the constraint). *Example:* Minimise $x^2 + y^2$ subject to $x + y = 4$: $\mathcal{L} = x^2 + y^2 + \lambda(x + y - 4)$; $2x + \lambda = 0$, $2y + \lambda = 0$, $x + y = 4$ give $x = y = -\lambda/2$, so $x = y = 2$ ($\lambda = -4$), $f = 8$.

Inequality constraints: KKT conditions. With $\mathcal{L} = f + \sum \mu_i g_i$ and $g_i \leq 0$, an optimum satisfies: (1) **stationarity** $\nabla \mathcal{L} = \mathbf{0}$; (2) **primal feasibility** $g_i \leq 0$; (3) **dual feasibility** $\mu_i \geq 0$; (4) **complementary slackness** $\mu_i g_i = 0$ (either the constraint is active, $g_i = 0$, or its multiplier is zero). *Example:* Minimise $x^2 + y^2$ subject to $x + y \geq 1$, i.e. $g = 1 - x - y \leq 0$. Stationarity: $x = y = \mu/2$. Case $\mu = 0$ gives $(0, 0)$, which violates the constraint, so reject. Case $g = 0$: $\mu = 1$, $x = y = 0.5$, $f = 0.5$, and $\mu \geq 0$ holds. With the constraint $x + y \geq -3$ instead, $\mu = 0$ works and the origin is optimal: the constraint is inactive.

Convex optimisation. A set C is *convex* if $t\mathbf{x} + (1-t)\mathbf{y} \in C$ for all $\mathbf{x}, \mathbf{y} \in C$, $t \in [0, 1]$ (a disk is convex). A

function is *convex* if $f(tx + (1-t)y) \leq tf(x) + (1-t)f(y)$ (the chord lies above the curve); equivalently H is positive semidefinite everywhere. *Example:* x^2 at 1, 3, $t = 0.5$: $f(2) = 4 \leq \frac{1}{2}(1) + \frac{1}{2}(9) = 5$; x^3 at $-2, 1$: $f(-0.5) = -0.125 > -3.5$, so x^3 is not convex. **Key property:** for a convex problem every local minimum is a global minimum, so gradient descent converges to the best solution. Linear and logistic regression and SVMs are convex; deep neural network losses are not, which is why their training needs care (momentum, good initialisation).

4.4 Review of the Machine Learning Pipeline

A pipeline is the chain of stages that takes data from collection to a deployed, monitored model.

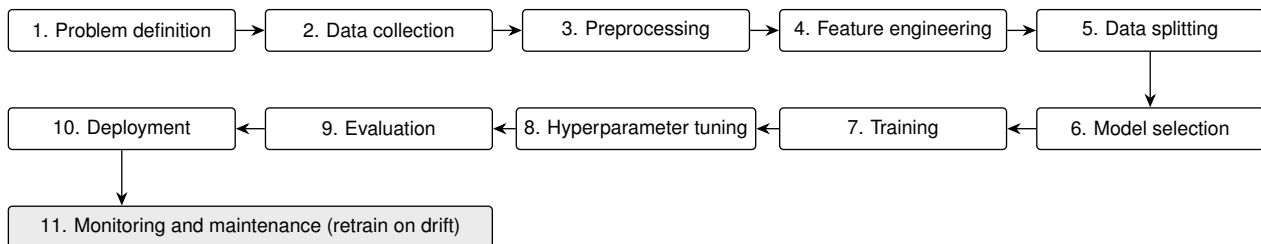


Figure 20: The eleven stages of the ML pipeline.

Stage	Main activity
Problem definition	State the task (classification, regression, clustering), the available data and the success measure.
Data collection	Databases, files, sensors, APIs, web; watch for missing, noisy, inconsistent or insufficient data.
Preprocessing	Handle missing values (drop, mean or median fill), remove duplicates, encode categories (Yes/No to 1/0), scale features, exploratory analysis.
Feature engineering	Create (BMI = weight/height ²), select (correlation, chi square) and reduce (PCA) features.
Splitting	Training set (fit), validation set (tune), test set (final unbiased check), for example 70/15/15.
Model selection and training	Choose an algorithm by data size, complexity, accuracy needs, cost and interpretability; minimise the loss on training data.
Hyperparameter tuning	Search learning rate, depth, number of layers, batch size by validation performance.
Evaluation	Accuracy, precision, recall, F1 on the test set.
Deployment and monitoring	Serve through web, mobile, cloud or embedded systems; watch for <i>data drift</i> (input distribution or input output relation changes) and retrain.

Model development: generative versus discriminative

A classifier estimates $P(Y | X)$ or a mapping $f : X \rightarrow Y$. A **generative** model learns the joint distribution, that is $P(X | Y)$ and $P(Y)$, and obtains the posterior with Bayes theorem. A **discriminative** model learns $P(Y | X)$ (or the boundary) directly.

	Generative	Discriminative
Learns	Distribution of data in each class, $P(X, Y)$	Decision boundary, $P(Y X)$
Can generate new data	Yes	No
Data need	Works with smaller data; stronger assumptions	Usually needs more labelled data; often more accurate
Missing data	Handles well	Poorly
Examples	Naive Bayes, Bayesian networks, HMM, GAN, VAE	Logistic regression, SVM, decision tree, neural network

Example: Naive Bayes spam filter (generative). Of 100 emails, 40 are spam; “free” appears in 30 spam and 5 of the 60 non spam. $P(\text{free} \mid \text{spam}) = 0.75$, $P(\text{free} \mid \text{not}) = 0.083$. Then

$$P(\text{spam} \mid \text{free}) = \frac{0.75(0.4)}{0.75(0.4) + 0.083(0.6)} = \frac{0.30}{0.35} = 0.857.$$

A discriminative logistic regression would instead learn $P(\text{spam} \mid x) = \sigma(w \cdot x + b)$ directly and predict spam when it exceeds 0.5.

Learning algorithm

Inductive learning finds general patterns or rules from examples; *deductive* learning derives specific facts from general rules. By feedback available, learning is:

Type	Data and goal	Examples
Supervised	Labelled (x, y) ; learn the mapping $x \rightarrow y$. Classification (discrete) or regression (continuous).	Spam filter, house price, fraud detection
Unsupervised	Unlabelled x ; find structure.	Clustering, dimensionality reduction, anomaly detection
Semi supervised	A few labelled plus many unlabelled examples.	Web page or image classification
Reinforcement	An agent acts, receives rewards; learn a policy that maximises total reward.	Game playing, robot control

Capacity, overfitting and underfitting

The aim is not low training error but low error on *new* data drawn from the same distribution, called **generalisation**. **Capacity** is the range of functions a model can fit: low (a straight line) versus high (a degree 9 polynomial, a deep network).

- **Underfitting:** model too simple; high training error and high test error (high bias).
- **Overfitting:** model memorises noise; very low training error but high test error (high variance).

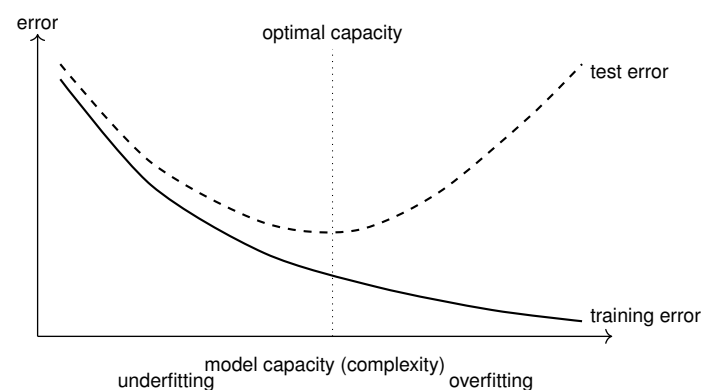


Figure 21: Training error keeps falling with capacity; test error is U shaped.

Polynomial degree	1	2 to 4	8
Training accuracy	68%	88% to 96%	99%
Test accuracy	65%	86% to 94%	75%
Diagnosis	Underfit	Good fit	Overfit

Remedies. Overfitting: more data, simpler model, regularisation (L1, L2), dropout, early stopping, cross validation, feature selection. Underfitting: more expressive model, more or better features, less regularisation, longer training.

Hyperparameters and validation sets

Parameters (weights, biases) are learned from training data. **Hyperparameters** (learning rate, number of layers, batch size, tree depth, regularisation strength, k in a nearest neighbour model) are set before training and control learning; they are not learned from the training data, so they are tuned on a **validation set**, which is used to compare models, choose hyperparameters and stop early, but never to update parameters. The **test set** is used once at the end. Search methods: grid search and random search.

k fold cross validation. With little data, split it into k equal folds; train on $k - 1$ folds and validate on the remaining one; repeat k times so every fold is used once for validation, and average: $CV = \frac{1}{k} \sum_{i=1}^k \text{score}_i$. *Example:* five fold accuracies 0.82, 0.85, 0.80, 0.84, 0.86 give $CV = 0.834$.

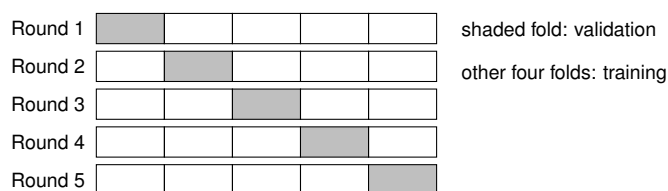


Figure 22: Five fold cross validation.

Estimators, bias and variance

An **estimator** is a rule that computes an estimate of an unknown parameter from data; the rule is the estimator, the number it gives is the estimate (sample mean $\bar{x} = \frac{1}{n} \sum x_i$ of 70, 75, 80, 85, 90 is 80). A trained ML model is itself an estimator of the true function. For an estimator $\hat{\theta}$ of θ :

$$\text{Bias}(\hat{\theta}) = E[\hat{\theta}] - \theta, \quad \text{Var}(\hat{\theta}) = E[(\hat{\theta} - E[\hat{\theta}])^2], \quad \text{MSE} = \text{Bias}^2 + \text{Var} (+\text{noise}).$$

Bias measures systematic error from over simple assumptions (accuracy); **variance** measures sensitivity to the particular training sample (consistency). *Example:* True mean salary 50,000; estimates 46,000, 47,000, 48,000, 47,000, 46,000 average 46,800, so bias = $-3,200$: consistently low. Heights estimated as 169, 170, 171, 170, 170 (low variance) versus 155, 180, 172, 165, 178 (high variance).

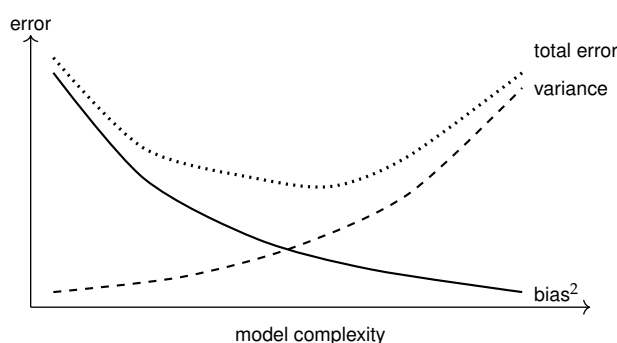


Figure 23: Bias variance tradeoff: bias² (solid) falls, variance (dashed) rises, total (dotted) is minimal in between.

Model	Bias	Variance
Underfitting	High	Low
Good fit	Moderate	Moderate
Overfitting	Low	High

MLE, MAP and cross entropy

Likelihood $L(\theta) = P(D \mid \theta)$ treats the data as fixed and the parameter as unknown (probability is the reverse: parameters fixed, data unknown). **Maximum likelihood estimation (MLE)** picks $\hat{\theta}_{\text{MLE}} = \arg \max_{\theta} P(D \mid \theta)$, in practice by maximising the log likelihood.

Example: A coin gives 8 heads and 2 tails in 10 tosses. $L(\theta) = \theta^8(1-\theta)^2$, $\ell = 8\ln\theta + 2\ln(1-\theta)$, $\ell' = \frac{8}{\theta} - \frac{2}{1-\theta} = 0$ gives $\hat{\theta}_{\text{MLE}} = 0.8$ (in general k/n). Values: $L(0.5) = 0.00098$, $L(0.7) = 0.00545$, $L(0.8) = 0.00671$, $L(0.9) = 0.00387$.

Maximum a posteriori (MAP) adds a prior $P(\theta)$ and maximises the posterior by Bayes theorem:

$$\hat{\theta}_{\text{MAP}} = \arg \max_{\theta} P(\theta | D) = \arg \max_{\theta} P(D | \theta) P(\theta),$$

since $P(D)$ does not depend on θ . With a uniform prior MAP equals MLE. *Example:* With a Beta(2,2) prior (belief that the coin is roughly fair), $\hat{\theta}_{\text{MAP}} = \frac{k+\alpha-1}{n+\alpha+\beta-2} = \frac{8+1}{10+2} = 0.75$, pulled toward 0.5. A Gaussian prior on weights gives L2 regularisation. MAP is less prone to overfitting on small data.

Entropy and cross entropy. Entropy $H(P) = -\sum_i P(i) \log P(i)$ measures uncertainty (a fair coin has the maximum, a certain outcome zero). Cross entropy measures how far a predicted distribution q is from the true one p :

$$H(p, q) = -\sum_i p_i \log q_i, \quad \text{binary: } -[y \ln \hat{y} + (1-y) \ln(1-\hat{y})].$$

Minimising cross entropy is the same as maximising the likelihood, so it is the standard classification loss. *Example:* True class Cat; predicted $P(\text{Cat}) = 0.90$: loss $-\ln 0.90 = 0.105$; at 0.40: 0.916; at 0.01 (confidently wrong): 4.605. It penalises confident mistakes heavily, unlike plain accuracy.

4.5 Supervised Learning: Decision Tree and Support Vector Machine

Decision tree (ID3)

A decision tree classifies an example by a series of tests. **Internal nodes** test an attribute, **branches** are the test outcomes, and **leaves** give the class. **ID3** (Quinlan) builds the tree top down, at each node choosing the attribute with the highest *information gain*.

Entropy and information gain

For a set S with class proportions p_i : Entropy(S) = $-\sum_i p_i \log_2 p_i$ (0 if pure; 1 for a 50/50 binary split).

$$\text{Gain}(S, A) = \text{Entropy}(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \text{Entropy}(S_v).$$

Algorithm. (1) If all examples are of one class, make a leaf. (2) Otherwise compute the gain of every remaining attribute and split on the best. (3) Partition the examples by its values and repeat on each subset until every subset is pure or attributes run out. Deep trees overfit, so they are pruned.

Example: Play golf (14 days; 9 Yes, 5 No).

D	Outlook	Temp	Humidity	Wind	Play	D	Outlook	Temp	Humidity	Wind	Play
1	Sunny	Hot	High	Weak	No	8	Sunny	Mild	High	Weak	No
2	Sunny	Hot	High	Strong	No	9	Sunny	Cool	Normal	Weak	Yes
3	Overcast	Hot	High	Weak	Yes	10	Rain	Mild	Normal	Weak	Yes
4	Rain	Mild	High	Weak	Yes	11	Sunny	Mild	Normal	Strong	Yes
5	Rain	Cool	Normal	Weak	Yes	12	Overcast	Mild	High	Strong	Yes
6	Rain	Cool	Normal	Strong	No	13	Overcast	Hot	Normal	Weak	Yes
7	Overcast	Cool	Normal	Strong	Yes	14	Rain	Mild	High	Strong	No

Entropy(S) = $-\frac{9}{14} \log_2 \frac{9}{14} - \frac{5}{14} \log_2 \frac{5}{14} = 0.940$. Gain for each attribute (counts as Yes/No):

Attribute	Subsets and entropies	Weighted	Gain
Outlook	Sunny (2/3): 0.971; Overcast (4/0): 0; Rain (3/2): 0.971	0.694	0.247
Humidity	High (3/4): 0.985; Normal (6/1): 0.592	0.789	0.151
Wind	Weak (6/2): 0.811; Strong (3/3): 1.000	0.892	0.048
Temperature	Hot (2/2): 1.0; Mild (4/2): 0.918; Cool (3/1): 0.811	0.911	0.029

Outlook has the highest gain and becomes the root. Overcast is pure (Yes). For Sunny (D1, D2, D8, D9, D11) Humidity separates the classes perfectly (High: No, Normal: Yes); for Rain (D4, D5, D6, D10, D14) Wind does (Weak: Yes, Strong: No).

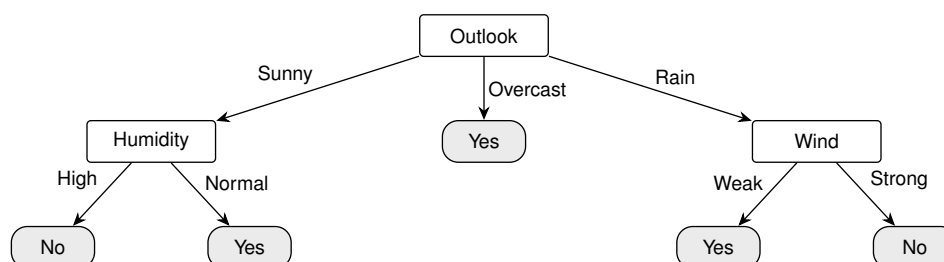


Figure 24: Decision tree learned by ID3 from the play golf data.

Rules read off the tree: IF Outlook is Overcast THEN Yes; IF Sunny AND Humidity is High THEN No; and so on.

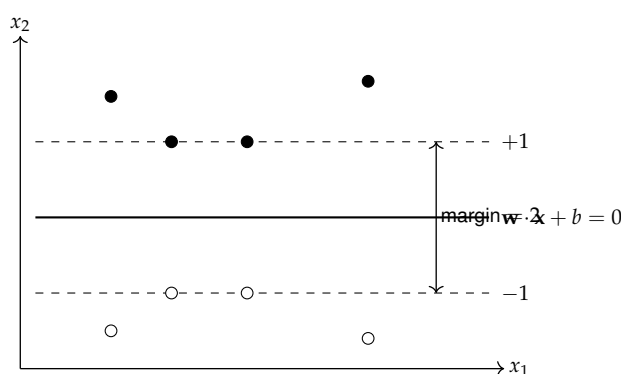
Support vector machine (SVM)

Support vector machine

An SVM finds the **hyperplane** $\mathbf{w} \cdot \mathbf{x} + b = 0$ that separates two classes with the **maximum margin**. Classifier: $f(\mathbf{x}) = \text{sign}(\mathbf{w} \cdot \mathbf{x} + b)$. The **support vectors** are the points closest to the hyperplane, on the margin lines $\mathbf{w} \cdot \mathbf{x} + b = \pm 1$; only they determine the boundary. The margin width is $\frac{2}{\|\mathbf{w}\|}$, so maximising margin means

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{subject to} \quad y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1, \quad y_i \in \{-1, +1\}.$$

Hard margin: data must be perfectly separable, no violations. **Soft margin:** allows some violations using slack $\xi_i \geq 0$ and minimises $\frac{1}{2} \|\mathbf{w}\|^2 + C \sum \xi_i$; large C punishes errors (narrow margin), small C tolerates them (wide margin). **Kernel trick:** for data that is not linearly separable, a kernel $K(\mathbf{x}, \mathbf{x}')$ acts as a dot product in a higher dimensional space without computing it: linear $\mathbf{x} \cdot \mathbf{x}'$; polynomial $(\mathbf{x} \cdot \mathbf{x}' + 1)^d$; RBF $e^{-\gamma \|\mathbf{x} - \mathbf{x}'\|^2}$.



filled: class +1, open: class -1; points on the dashed lines are support vectors

Figure 25: Maximum margin hyperplane for the data of Example 1.

Example: 1. Class +1: $A(2,3), B(3,3)$; class -1: $C(2,1), D(3,1)$. The classes are separated horizontally, so the boundary is $x_2 = 2$, i.e. $x_2 - 2 = 0$: $\mathbf{w} = (0,1)$, $b = -2$, $f(\mathbf{x}) = x_2 - 2$. Check: $f(A) = 1$, $f(B) = 1$, $f(C) = -1$, $f(D) = -1$, all correct and on the margins. Margin $= 2/\|\mathbf{w}\| = 2$. New point $P(4,2.8)$: $f = 0.8 > 0$, class +1; $Q(5,1.3)$: $f = -0.7$, class -1.

Example: 2 (solving for support vectors). Support vectors: $s_1 = (1,0)$ negative; $s_2 = (3,1)$ and $s_3 = (3,-1)$ positive. Append a bias input 1: $\tilde{s}_1 = (1,0,1)$, $\tilde{s}_2 = (3,1,1)$, $\tilde{s}_3 = (3,-1,1)$. Solve $\sum_j \alpha_j \tilde{s}_j \cdot \tilde{s}_i = y_i$ ($-1, +1, +1$):

$$2\alpha_1 + 4\alpha_2 + 4\alpha_3 = -1, \quad 4\alpha_1 + 11\alpha_2 + 9\alpha_3 = 1, \quad 4\alpha_1 + 9\alpha_2 + 11\alpha_3 = 1$$

gives $\alpha_1 = -3.5$, $\alpha_2 = \alpha_3 = 0.75$. Then $\tilde{\mathbf{w}} = \sum \alpha_j \tilde{s}_j = (1,0,-2)$, so $\mathbf{w} = (1,0)$, $b = -2$: hyperplane $x_1 = 2$, margin $2/\|\mathbf{w}\| = 2$.

4.6 Unsupervised Learning: t-SNE

t-distributed Stochastic Neighbour Embedding (t-SNE) is a nonlinear dimensionality reduction method used mainly to *visualise* high dimensional data in 2D or 3D. It preserves **local neighbourhood structure**: points that are neighbours in high dimensions should stay neighbours in the map. Unlike PCA, which keeps global variance with a linear projection, t-SNE keeps neighbour probabilities.

Steps.

1. *High dimensional similarities.* A Gaussian centred at $\mathbf{x}_i$ gives the probability that $\mathbf{x}_i$ picks $\mathbf{x}_j$ as a neighbour, then it is symmetrised over n points:

$$p_{j|i} = \frac{e^{-\|\mathbf{x}_i - \mathbf{x}_j\|^2 / 2\sigma_i^2}}{\sum_{k \neq i} e^{-\|\mathbf{x}_i - \mathbf{x}_k\|^2 / 2\sigma_i^2}}, \quad p_{ij} = \frac{p_{j|i} + p_{i|j}}{2n}.$$

σ_i is set from the chosen **perplexity** (roughly the effective number of neighbours, typically 5 to 50).

2. *Low dimensional similarities.* Map points to $\mathbf{y}_i$ (2D or 3D) and use a Student t distribution with one degree of freedom, whose heavy tails avoid the *crowding problem*:

$$q_{ij} = \frac{(1 + \|\mathbf{y}_i - \mathbf{y}_j\|^2)^{-1}}{\sum_{k \neq l} (1 + \|\mathbf{y}_k - \mathbf{y}_l\|^2)^{-1}}.$$

3. *Cost.* Kullback Leibler divergence $C = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}}$.
4. *Optimise* by gradient descent: $\frac{\partial C}{\partial \mathbf{y}_i} = 4 \sum_j (p_{ij} - q_{ij})(\mathbf{y}_i - \mathbf{y}_j)(1 + \|\mathbf{y}_i - \mathbf{y}_j\|^2)^{-1}$, and $\mathbf{y}_i \leftarrow \mathbf{y}_i - \eta \frac{\partial C}{\partial \mathbf{y}_i}$.

Example: Marks (Maths, Physics): $A(50,55)$, $B(52,53)$, $C(90,92)$ with $\sigma = 10$. $\|A - B\|^2 = 8$, $\|A - C\|^2 = 2969$, so $p_{B|A} = \frac{e^{-0.04}}{e^{-0.04} + e^{-14.8}} \approx 1$ and $p_{C|A} \approx 0$: in the 1D map A and B are placed close together and C far away. **Cautions:** low perplexity stresses tiny clusters, high perplexity global layout; cluster sizes and distances between clusters in the picture are not reliable; runs are random and slow on large data.

4.7 Semi Supervised and Reinforcement Learning

Semi supervised learning trains on a *small labelled set* (X_l, y_l) and a *large unlabelled set* X_u , aiming to beat a classifier trained on the labelled data alone. **Example:** 20 labelled and 5000 unlabelled cat and dog images: the unlabelled images reveal the shape of the data. It relies on assumptions: *smoothness* (nearby points share labels), *cluster* (points in a cluster share a label), *manifold* (data lie on a low dimensional surface); if they fail, unlabelled data can hurt.

Self training (simplest method): (1) train a classifier on the labelled set L ; (2) predict labels for unlabelled set U ; (3) keep only confident predictions (say probability > 0.9) as pseudo labels and add them to L ;

(4) retrain; (5) repeat until no confident samples remain. Benefits: lower labelling cost, better accuracy when labels are scarce.

Reinforcement learning (RL). An *agent* interacts with an *environment*: in state s it takes action a , receives reward r and moves to state s' . It learns a **policy** $\pi(s) \rightarrow a$ by trial and error to maximise cumulative (often discounted) reward $\sum_t \gamma^t r_t$, $0 \leq \gamma < 1$. The **value function** $V(s)$ is the expected future reward from s ; the **action value** $Q(s, a)$ adds the first action. The agent balances **exploration** (try new actions) and **exploitation** (use what is known), for example ϵ greedy. Rewards may be delayed. Uses: games, robotics, recommendation.

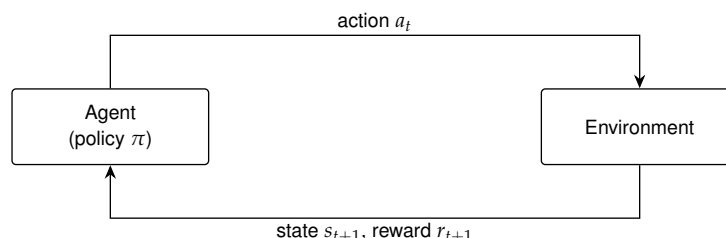


Figure 26: The reinforcement learning loop.

Q learning

Q learning is a *model free* (no need for transition probabilities) and *off policy* (learns the best policy while behaving otherwise) algorithm that learns a table $Q(s, a)$. Temporal difference update:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right].$$

With learning rate $\alpha = 1$ in a deterministic world this is the Bellman form $Q(s, a) = R(s, a) + \gamma \max_{a'} Q(s', a')$. After enough episodes, the best action in each state is $\arg \max_a Q(s, a)$.

Example: Rooms. Rooms 0 to 4 in a building; room 5 is outside (goal). Doors: 0 to 4, 4 to 3, 4 to 5, 3 to 1, 3 to 2, 1 to 5. Moving into room 5 gives reward 100, every other move 0; $\gamma = 0.8$; all Q start at 0.

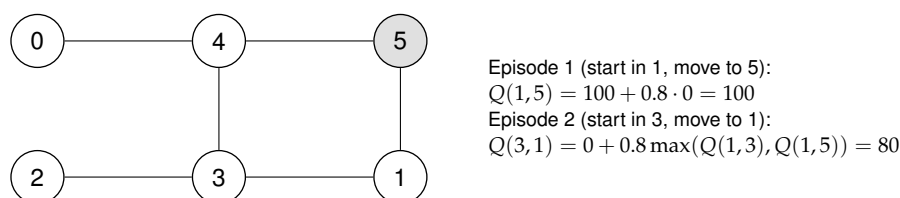


Figure 27: Rooms graph with the first two Q updates.

Continuing until convergence gives $Q(1, 5) = Q(4, 5) = 100$; $Q(3, 1) = Q(3, 4) = Q(0, 4) = 80$; $Q(1, 3) = Q(4, 3) = Q(4, 0) = Q(2, 3) = 64$; $Q(3, 2) = 51.2$. Following the largest Q from room 2: $2 \rightarrow 3 \rightarrow 1 \rightarrow 5$ (or via 4), the shortest route to the goal.

4.8 Model Evaluation: Energy Based Indicators

Accuracy alone is no longer enough: compute doubles roughly every few months, and cost, carbon emissions and latency matter. Three efficiency indicators are used alongside accuracy.

- **Training time or energy.** Time = end time – start time (affected by dataset size, epochs, batch size, hardware, parameters, optimiser). Energy = power $\times$ time. *Example:* 250 W GPU for 4 h: $250 \times 4 = 1000 \text{ Wh} = 1 \text{ kWh}$. Data centre cooling adds overhead.
- **Model size.** Number of parameters $\times$ bytes per parameter. *Example:* $2,000,000 \times 4 \text{ bytes} = 8 \text{ MB}$. Small models load fast, fit phones and IoT devices, and cost less to communicate and serve.
- **Inference time or energy.** Time for one prediction (*latency*; throughput is predictions per second); energy per prediction = power $\times$ time. *Example:* 30 W for 0.5 s: $30 \times 0.5/3600 = 0.0042 \text{ Wh}$.

Although one prediction is cheap, billions of them dominate lifetime energy; memory movement, not arithmetic, is often the main cost.

Example: Model A: accuracy 97%, 10 h at 250 W (2.5 kWh), 700 MB, 150 ms. Model B: 96%, 4 h at 200 W (0.8 kWh), 100 MB, 20 ms. B loses 1 point of accuracy but uses about a third of the energy, one seventh of the size and predicts 7 times faster, so it suits mobile and embedded use; parameter efficiency is now an evaluation metric.

Confusion matrix metrics (accuracy based evaluation). With TP, FP, FN, TN : $\text{accuracy} = \frac{TP+TN}{\text{all}}$, $\text{precision} = \frac{TP}{TP+FP}$, $\text{recall} = \frac{TP}{TP+FN}$, $F_1 = \frac{2PR}{P+R}$. *Example:* $TP = 40, FP = 10, FN = 5, TN = 45$: accuracy 0.85, precision 0.80, recall 0.889, $F_1 = 0.842$.

Chapter 5 Neural Networks and Deep Learning Algorithms

5.1 Neurons, Structures and Activation Functions

Biological and artificial neurons

A biological neuron receives signals through **dendrites**, combines them in the **soma** (cell body), and if the combined signal is strong enough it fires an electrical impulse along the **axon** (often insulated by a myelin sheath). The **synaptic terminals** pass the signal to the next cell by releasing neurotransmitters across the synapse. An **artificial neuron** is a mathematical function that weights its inputs, sums them with a bias and passes the result through an activation function.

Aspect	Biological neuron	Artificial neuron
Input, processing, output	Dendrites, soma, axon and synapse	Inputs x_i , weighted sum with bias, activation f
Strength of connection	Synaptic strength	Weight w_i (learned)
Signals	Chemical and electrical, analogue	Numbers, computed digitally
Speed and scale	Slow (milliseconds), about 10^{11} neurons, massively parallel	Very fast per operation, far fewer units, largely sequential hardware
Learning	Changes in synapses, poorly understood	Gradient based weight updates

Right brain and left brain analogy. The left brain stands for symbolic AI: logical, analytical, rule based (if then programs, expert systems, Chapters 2 and 3). The right brain stands for neural networks: intuitive, holistic and pattern oriented; deep learning senses patterns in data instead of following rigid rules.

A simple mathematical model of a neuron

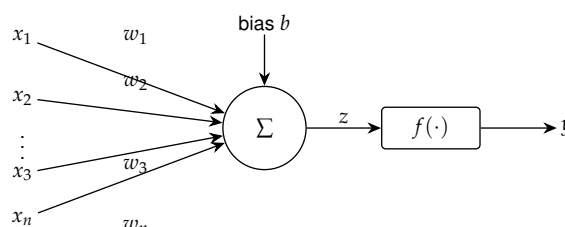


Figure 28: Artificial neuron: $z = \sum_i w_i x_i + b$, output $y = f(z)$.

Neuron model and MP neuron

$$z = \sum_{i=1}^n w_i x_i + b = \mathbf{w}^\top \mathbf{x} + b, \quad y = f(z).$$

The **McCulloch and Pitts (1943)** neuron is the first such model. Inputs are binary (0 or 1), weights are excitatory (+1) or inhibitory (veto), and the neuron fires ($y = 1$) if $\sum_i w_i x_i \geq \theta$ (threshold), else $y = 0$. It works in two stages, *aggregation* and *thresholding*, and showed that neurons can compute logic: AND uses $\theta = 2$ for two inputs, OR uses $\theta = 1$.

Structures and the universal approximation theorem

Neurons are arranged in layers. In a **feedforward** network signals flow from input layer through zero or more **hidden layers** to the output layer with no cycles; in a **recurrent** network outputs feed back as inputs (Section 5.4). The number of input units equals the number of features; output units equal the number of classes (classification) or target values (regression).

Why learning converges: universal approximation theorem. A feedforward network with a single hidden layer containing enough neurons and a non linear activation (such as the sigmoid) can approximate any continuous function on a bounded domain to any desired accuracy. This is why an MLP is called a universal function approximator. The theorem says such weights exist, not how to find them (backpropagation) nor how many neurons are needed.

Learning in neural networks: Hebbian learning and logic gates

Hebb rule

“Neurons that fire together wire together.” For inputs x_i and target t , the weights are strengthened when input and output agree:

$$w_i^{\text{new}} = w_i^{\text{old}} + x_i t, \quad b^{\text{new}} = b^{\text{old}} + t.$$

Start from $w = 0, b = 0$; use bipolar values $(+1, -1)$.

Example: AND gate (bipolar). Patterns $(x_1, x_2; t)$: $(1, 1; 1), (1, -1; -1), (-1, 1; -1), (-1, -1; -1)$.

x_1	x_2	t	Δw_1	Δw_2	Δb	w_1, w_2, b after
1	1	1	1	1	1	1, 1, 1
1	-1	-1	-1	1	-1	0, 2, 0
-1	1	-1	1	-1	-1	1, 1, -1
-1	-1	-1	1	1	-1	2, 2, -2

Final: $w_1 = w_2 = 2, b = -2$. Check $(1, 1)$: $2 + 2 - 2 = 2 > 0 \Rightarrow +1$; $(1, -1)$: $2 - 2 - 2 = -2 \Rightarrow -1$; $(-1, -1)$: $-2 - 2 - 2 = -6 \Rightarrow -1$. All four correct.

Activation functions: purpose and characteristics

The activation function makes the network *non linear*. Without it, any stack of layers collapses to one linear map $(W_2(W_1x) = (W_2W_1)x)$, and the universal approximation property is lost.

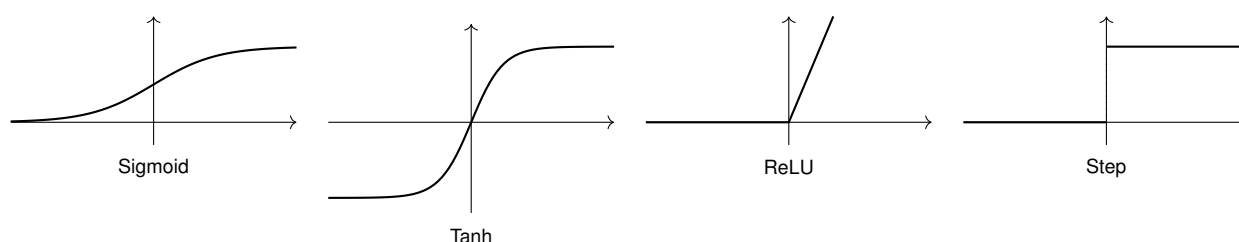


Figure 29: Common activation functions.

Function	Formula	Range	Characteristics and use
Step	1 if $z \geq 0$ else 0	$\{0, 1\}$	Perceptron, MP neuron. Not differentiable, so cannot be trained by gradients.
Sigmoid	$\sigma(z) = \frac{1}{1 + e^{-z}}$	$(0, 1)$	Smooth, read as probability, output layer of binary classifiers. $\sigma' = \sigma(1 - \sigma)$. Saturates for large $ z $ so gradients vanish.
Tanh	$\frac{e^z - e^{-z}}{e^z + e^{-z}}$	$(-1, 1)$	Zero centred, so usually better than sigmoid in hidden layers. Also saturates. $\tanh' = 1 - \tanh^2$.
ReLU	$\max(0, z)$	$[0, \infty)$	Cheap, no saturation for $z > 0$, default for hidden layers. Units with $z < 0$ can “die”.
Softmax	$\frac{e^{z_i}}{\sum_j e^{z_j}}$	$(0, 1)$, sums to 1	Output layer for multiclass classification.

5.2 Perceptron, Multilayer Perceptron and Backpropagation

Perceptron and its limitations

The **perceptron** (Rosenblatt) is a single neuron with a step activation, $y = \text{step}(\mathbf{w}^\top \mathbf{x} + b)$. It is a linear classifier: the boundary $\mathbf{w}^\top \mathbf{x} + b = 0$ is a line (hyperplane). It is a *single layer feedforward network* (inputs connected directly to output units).

Perceptron learning rule

For each training pair $(\mathbf{x}, t)$ with learning rate η and output y :

$$\mathbf{w} \leftarrow \mathbf{w} + \eta (t - y) \mathbf{x}, \quad b \leftarrow b + \eta (t - y).$$

Weights change only on a mistake. If the data are linearly separable the rule converges in a finite number of steps (perceptron convergence theorem).

Example: Learning AND (inputs 0 or 1, $\eta = 1$, $w = 0, b = 0$, output 1 if $z \geq 0$). Cycle through $(0,0), (0,1), (1,0), (1,1)$ with targets 0,0,0,1.

Epoch 1: $(0,0)$ gives $y = 1$, wrong: $b = -1$. $(1,1)$ gives $y = 0$, wrong: $w = (1,1), b = 0$.

Epoch 2: $(0,0)$ wrong: $b = -1$. $(0,1)$ wrong: $w = (1,0), b = -2$. $(1,1)$ wrong: $w = (2,1), b = -1$.

After a few more epochs the weights settle at $w = (2,1), b = -3$ with no mistakes: $(1,1)$ gives $z = 0 \Rightarrow 1$; $(1,0)$ gives -1 ; $(0,1)$ gives -2 ; $(0,0)$ gives -3 ; all correct.

Limitation. A perceptron cannot learn XOR, because no single line separates $\{(0,1), (1,0)\}$ from $\{(0,0), (1,1)\}$ (Minsky and Papert, 1969). It can only solve linearly separable problems. The remedy is a hidden layer.

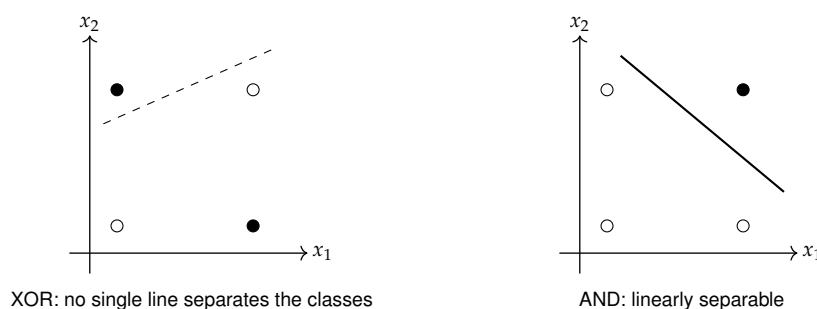


Figure 30: Linear separability (filled: output 1, open: output 0).

Example: XOR with one hidden layer (step units): $h_1 = \text{step}(x_1 + x_2 - 0.5)$ (OR), $h_2 = \text{step}(x_1 + x_2 - 1.5)$ (AND), $y = \text{step}(h_1 - h_2 - 0.5)$. Then $(0,0) \rightarrow (0,0) \rightarrow 0$; $(1,0) \rightarrow (1,0) \rightarrow 1$; $(0,1) \rightarrow (1,0) \rightarrow 1$; $(1,1) \rightarrow (1,1) \rightarrow -0.5 \rightarrow 0$. XOR is “OR but not AND”.

Adaline and the delta rule

Adaline (Widrow and Hoff) computes the linear output $z = \mathbf{w}^\top \mathbf{x} + b$ and learns by minimising the squared error of z (before the step) using the **delta rule** (least mean squares): $\mathbf{w} \leftarrow \mathbf{w} + \eta (t - z) \mathbf{x}$. Unlike the perceptron rule it updates on every example, even correct ones, and follows the gradient of $E = \frac{1}{2}(t - z)^2$. The delta rule is the ancestor of backpropagation.

Multilayer perceptron (MLP)

An **MLP** has an input layer, one or more hidden layers and an output layer, with every unit fully connected to the next layer and non linear activations in the hidden layers. Roles: *classification* with non linear boundaries, *regression*, and *feature representation* (hidden layers transform non linearly separable data into a space where it is linearly separable).

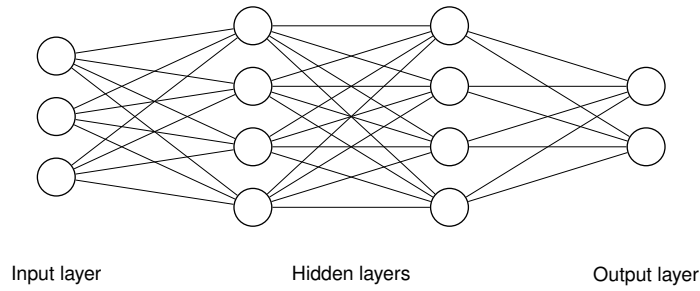


Figure 31: A fully connected MLP with two hidden layers.

Backpropagation algorithm and its derivation

Backpropagation computes the gradient of the loss with respect to every weight by applying the chain rule from output to input, and then gradient descent updates the weights. One iteration: (1) *forward pass*: compute all z and activations, then the loss L ; (2) *backward pass*: compute $\partial L / \partial(\cdot)$ layer by layer, reusing values from later layers; (3) *update*: $w \leftarrow w - \eta \partial L / \partial w$.

Derivation. For layer l let $\mathbf{z}^{[l]} = W^{[l]} \mathbf{a}^{[l-1]} + \mathbf{b}^{[l]}$ and $\mathbf{a}^{[l]} = f(\mathbf{z}^{[l]})$. Define the error signal $\delta^{[l]} = \partial L / \partial \mathbf{z}^{[l]}$. By the chain rule:

$$\delta^{[l]} = ((W^{[l+1]})^\top \delta^{[l+1]}) \odot f'(\mathbf{z}^{[l]}), \quad \frac{\partial L}{\partial W^{[l]}} = \delta^{[l]} (\mathbf{a}^{[l-1]})^\top, \quad \frac{\partial L}{\partial \mathbf{b}^{[l]}} = \delta^{[l]}.$$

For the output layer with a sigmoid and binary cross entropy $L = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$, the terms $\partial L / \partial \hat{y} = -\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}}$ and $\hat{y}' = \hat{y}(1 - \hat{y})$ multiply to the simple result $\delta^{[L]} = \hat{y} - y$. For squared error with a sigmoid it is $(\hat{y} - y)\hat{y}(1 - \hat{y})$.

Example: Numerical example. One input, two sigmoid hidden neurons, one sigmoid output, binary cross entropy. $x = 0.5, y = 1; w_{11} = 0.8, b_{11} = 0.2; w_{12} = -0.4, b_{12} = 0.5; w_{21} = 0.6, w_{22} = -0.5, b_2 = 0.1; \eta = 0.1$.

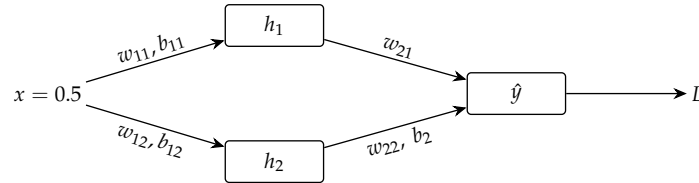


Figure 32: Network used in the backpropagation example.

Forward. $z_{11} = 0.8(0.5) + 0.2 = 0.6, h_1 = \sigma(0.6) = 0.6457. z_{12} = -0.4(0.5) + 0.5 = 0.3, h_2 = \sigma(0.3) = 0.5744. z_2 = 0.6(0.6457) - 0.5(0.5744) + 0.1 = 0.2002, \hat{y} = \sigma(0.2002) = 0.5499. \text{ Loss } L = -\ln(0.5499) = 0.5981.$

Backward. $\delta_2 = \hat{y} - y = -0.4501.$

Gradient	Working	Value
$\partial L / \partial w_{21}$	$\delta_2 h_1 = -0.4501(0.6457)$	-0.2906
$\partial L / \partial w_{22}$	$\delta_2 h_2 = -0.4501(0.5744)$	-0.2586
$\partial L / \partial b_2$	δ_2	-0.4501
$\partial L / \partial h_1$	$\delta_2 w_{21} = -0.4501(0.6)$	-0.2701
$\partial L / \partial h_2$	$\delta_2 w_{22} = -0.4501(-0.5)$	0.2251
$\delta_{11} = \partial L / \partial z_{11}$	$-0.2701 \cdot h_1(1 - h_1) = -0.2701(0.2288)$	-0.0618
$\delta_{12} = \partial L / \partial z_{12}$	$0.2251 \cdot h_2(1 - h_2) = 0.2251(0.2445)$	0.0550
$\partial L / \partial w_{11}, \partial L / \partial b_{11}$	$\delta_{11} x, \delta_{11}$	-0.0309, -0.0618
$\partial L / \partial w_{12}, \partial L / \partial b_{12}$	$\delta_{12} x, \delta_{12}$	0.0275, 0.0550

Update ($\eta = 0.1$): $w_{21} = 0.6 + 0.0291 = 0.6291$; $w_{22} = -0.5 + 0.0259 = -0.4741$; $b_2 = 0.1 + 0.0450 = 0.1450$; $w_{11} = 0.8031$; $b_{11} = 0.2062$; $w_{12} = -0.4028$; $b_{12} = 0.4945$. The output for the same input will now be slightly closer to 1.

5.3 Introduction to Deep Learning

Machine learning versus deep learning; deep versus traditional networks

Feature	Machine learning	Deep learning
Features	Hand crafted by experts (feature engineering)	Learned automatically from raw data
Data needed	Works with small and medium data	Needs large data to perform well
Compute	Modest, CPU is usually enough	Heavy, GPUs or TPUs
Interpretability	Often easier to explain (trees, linear models)	Black box, harder to explain
Training time	Short	Long

Deep learning uses neural networks with *many* hidden layers (a “deep” network, typically more than a few), whereas a traditional (shallow) network has one or two. Depth lets the network build a hierarchy of features.

Why deep representations? Each layer builds on the one before: in an image network, early layers detect edges, middle layers combine edges into parts (eyes, wheels), and later layers recognise whole objects; in speech, from sounds to phonemes to words. A shallow network would need exponentially many units to represent the same functions that a deep one represents compactly.

Building blocks, forward and backward propagation

A deep network of L layers repeats one block per layer. **Forward block** for layer l : input $\mathbf{a}^{[l-1]}$, compute $\mathbf{z}^{[l]} = W^{[l]}\mathbf{a}^{[l-1]} + \mathbf{b}^{[l]}$, $\mathbf{a}^{[l]} = f^{[l]}(\mathbf{z}^{[l]})$, and *cache* $\mathbf{z}^{[l]}$. **Backward block**: input $d\mathbf{a}^{[l]}$, use the cache, output $d\mathbf{a}^{[l-1]}$, $dW^{[l]}$, $d\mathbf{b}^{[l]}$; then update $W^{[l]} \leftarrow W^{[l]} - \eta dW^{[l]}$.

Vectorised implementation and matrix dimensions

Processing all m examples at once with matrices replaces slow loops. Stack examples as columns: $X = A^{[0]} \in \mathbb{R}^{n_0 \times m}$, with n_l units in layer l .

$$\begin{aligned}
 \text{Forward: } \quad & \mathbf{Z}^{[l]} = W^{[l]}A^{[l-1]} + \mathbf{b}^{[l]}, & A^{[l]} &= f(\mathbf{Z}^{[l]}) \\
 \text{Backward: } \quad & d\mathbf{Z}^{[l]} = dA^{[l]} \odot f'(\mathbf{Z}^{[l]}), & dW^{[l]} &= \frac{1}{m} d\mathbf{Z}^{[l]}A^{[l-1]\top} \\
 & d\mathbf{b}^{[l]} = \frac{1}{m} \sum_{\text{columns}} d\mathbf{Z}^{[l]}, & dA^{[l-1]} &= W^{[l]\top} d\mathbf{Z}^{[l]}
 \end{aligned}$$

Quantity	Shape	Reason
$W^{[l]}$ and $dW^{[l]}$	$n_l \times n_{l-1}$	maps n_{l-1} inputs to n_l outputs
$\mathbf{b}^{[l]}$ and $d\mathbf{b}^{[l]}$	$n_l \times 1$	one bias per unit (broadcast over m columns)
$\mathbf{Z}^{[l]}, A^{[l]}, d\mathbf{Z}^{[l]}, dA^{[l]}$	$n_l \times m$	one column per example

Example: Input has $n_0 = 3$ features, hidden layer 1 has $n_1 = 4$, layer 2 has $n_2 = 2$, and $m = 5$: $W^{[1]}$ is 4×3 , $\mathbf{Z}^{[1]}$ is 4×5 , $W^{[2]}$ is 2×4 , $\mathbf{Z}^{[2]}$ is 2×5 . *Check rule:* the inner dimensions of every product must match, and W and dW must have equal shapes.

Prerequisites for successful deep learning

Deep learning needs: (1) **large amounts of good quality, labelled data**; (2) **powerful compute** (GPUs, TPUs); (3) **good algorithms and architectures** (ReLU, better optimisers, regularisation, batch

normalisation, sensible initialisation); (4) **careful tuning** of hyperparameters and validation to avoid overfitting; (5) suitable software frameworks and enough storage and memory.

5.4 Concepts on Convolutional, Recurrent and Generative Networks

Convolutional neural networks (CNN)

A CNN is a feedforward network designed for grid like data such as images. A fully connected layer on a $224 \times 224 \times 3$ image would need over 150,000 weights per neuron. A CNN instead uses small *local* filters whose weights are *shared* across the whole image, which cuts parameters sharply and makes detection of a feature independent of where it appears (translation equivariance).

Components.

- **Convolution layer.** Slides a small *kernel* (filter) over the input; at each position it takes the sum of element wise products. Many kernels give many *feature maps* (channels). Deep learning libraries actually compute cross correlation (no kernel flip): $O(i, j) = \sum_m \sum_n I(i + m, j + n) K(m, n)$.
- **Padding p** adds a border of zeros so the output does not shrink and edge pixels are used fully. **Stride s** is the step of the kernel; a larger stride downsamples.
- **Activation** (usually ReLU) after each convolution.
- **Pooling layer.** Downsamples each feature map, e.g. *max pooling* keeps the largest value in each window (commonly 2×2 , stride 2). It has no weights and gives small shift invariance.
- **Fully connected layers and softmax** at the end for classification.

Output size

For input size n , kernel size f , padding p , stride s :

$$n_{\text{out}} = \left\lfloor \frac{n + 2p - f}{s} \right\rfloor + 1.$$

A convolution layer with k kernels of size $f \times f$ on c input channels has $k(f^2c + 1)$ parameters (one bias per kernel).

Example: Convolution. Input $I = \begin{pmatrix} 10 & 20 & 30 \\ 40 & 50 & 60 \\ 70 & 80 & 90 \end{pmatrix}$, kernel $K = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$, $p = 0$, $s = 1$, so the output is 2×2 (since $(3 - 2)/1 + 1 = 2$).

Input 3×3		Kernel 2×2		Output 2×2																		
<table><tr><td>10</td><td>20</td><td>30</td></tr><tr><td>40</td><td>50</td><td>60</td></tr><tr><td>70</td><td>80</td><td>90</td></tr></table>	10	20	30	40	50	60	70	80	90	*	<table><tr><td>1</td><td>2</td></tr><tr><td>3</td><td>4</td></tr></table>	1	2	3	4	=	<table><tr><td>370</td><td>470</td></tr><tr><td>670</td><td>770</td></tr></table>	370	470	670	770	$O_{11} = 10(1) + 20(2) + 40(3) + 50(4) = 370$ $O_{12} = 20 + 60 + 150 + 240 = 470$ $O_{21} = 40 + 100 + 210 + 320 = 670$ $O_{22} = 50 + 120 + 240 + 360 = 770$
10	20	30																				
40	50	60																				
70	80	90																				
1	2																					
3	4																					
370	470																					
670	770																					

Figure 33: Cross correlation of a 3×3 input with a 2×2 kernel.

Example: Max pooling (2×2 , stride 2) on $\begin{pmatrix} 1 & 3 & 2 & 1 \\ 4 & 6 & 5 & 0 \\ 7 & 2 & 9 & 8 \\ 1 & 0 & 3 & 4 \end{pmatrix}$ gives $\begin{pmatrix} 6 & 5 \\ 7 & 9 \end{pmatrix}$.

LeNet 5 (digit recognition, 32×32 grey image), a classic CNN:

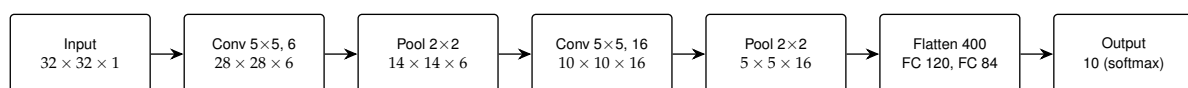


Figure 34: LeNet 5 layer shapes.

Parameters: C1 $6(25 + 1) = 156$; C3 $16(6 \cdot 25 + 1) = 2416$; FC 120: $120(400 + 1) = 48,120$; FC 84: $84(121) = 10,164$; output: $10(85) = 850$; total 61,706. Pooling layers add none.

Backpropagation in a convolution layer. The gradient with respect to a kernel weight is the cross correlation of the layer input with the upstream gradient: $\frac{\partial L}{\partial K} = I \star \frac{\partial L}{\partial O}$ (each kernel weight collects the input values it multiplied, weighted by the output gradients). The gradient passed back to the input is the full convolution of $\partial L / \partial O$ with the 180 degree rotated kernel. For max pooling the gradient goes only to the position that held the maximum.

*Applications** (image classification, object detection, medical imaging) are descriptive only.

Recurrent neural networks (RNN)

An RNN processes sequences (text, speech, time series) by keeping a *hidden state* that remembers the past. The same weights are reused at every time step. Feedforward networks assume independent inputs of fixed size; RNNs handle variable length, ordered data.

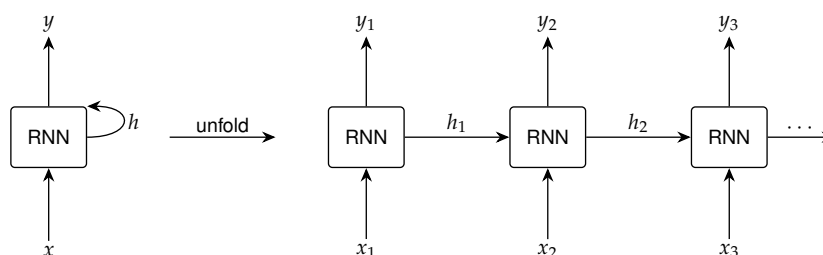


Figure 35: An RNN (left) and the same network unfolded over time (right); the weights are shared across steps.

Vanilla RNN equations

$$\mathbf{h}_t = \tanh(W_{hh}\mathbf{h}_{t-1} + W_{xh}\mathbf{x}_t + \mathbf{b}_h), \quad \mathbf{y}_t = W_{hy}\mathbf{h}_t + \mathbf{b}_y.$$

$\mathbf{h}_0 = \mathbf{0}$. The loss of a sequence is the sum of the step losses, $L = \sum_t L_t$. Training uses **backpropagation through time (BPTT)**: unfold the network and backpropagate through all steps, adding the gradients of the shared weights.

Components. A *recurrent neuron* feeds its output back to itself at the next step; a *recurrent layer* is a group of such neurons with input weights W_{xh} , recurrent weights W_{hh} and output weights W_{hy} .

Types by input and output shape. One to many (image captioning), many to one (sentiment classification), many to many with equal length (tagging each word), many to many with different lengths (machine translation, encoder decoder).

Problem: vanishing and exploding gradients. BPTT multiplies the gradient by W_{hh} and $\tanh'$ once per step. Over long sequences the product shrinks toward zero (vanishing, the network forgets distant inputs) or blows up (exploding, fixed by gradient clipping). Gated units fix the first problem.

Variants. *Bidirectional RNN*: two RNNs read the sequence forwards and backwards and their states are joined, so each output sees past and future context.

LSTM and GRU

LSTM (long short term memory) adds a cell state $\mathbf{c}_t$ (a memory highway) controlled by three sigmoid gates $\in (0, 1)$ acting on $[\mathbf{h}_{t-1}, \mathbf{x}_t]$:

$$\begin{aligned} \mathbf{f}_t &= \sigma(W_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f), & \mathbf{i}_t &= \sigma(W_i[\cdot] + \mathbf{b}_i), & \mathbf{o}_t &= \sigma(W_o[\cdot] + \mathbf{b}_o), \\ \tilde{\mathbf{c}}_t &= \tanh(W_c[\cdot] + \mathbf{b}_c), & \mathbf{c}_t &= \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t, & \mathbf{h}_t &= \mathbf{o}_t \odot \tanh(\mathbf{c}_t). \end{aligned}$$

Forget gate $\mathbf{f}$ decides what to erase, input gate $\mathbf{i}$ what to write, output gate $\mathbf{o}$ what to expose. Because $\mathbf{c}_t$ is updated additively, gradients survive over long spans.

GRU (gated recurrent unit) is a simpler variant with two gates and no separate cell state: update gate $\mathbf{z}_t$ and reset gate $\mathbf{r}_t$, with $\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t$. It has fewer parameters and often performs similarly.

*Applications** (speech recognition, language modelling, translation, forecasting) are descriptive only.

History of generative models

A **generative model** learns the data distribution $P(\mathbf{x})$ (or $P(\mathbf{x}, y)$) so it can create new samples.

Period	Developments
Before 2000 (statistical)	Probability distributions: Markov chains, Naive Bayes, Gaussian mixture models (GMM) estimate the underlying data pattern.
2000s (energy based and graphical)	Restricted Boltzmann machines (RBM) and hidden Markov models (HMM), used in unsupervised learning, speech recognition and collaborative filtering.
2014	Variational autoencoders (VAE) and generative adversarial networks (GAN) transform computer vision.
2017 onward	The Transformer architecture; autoregressive language models (early GPT) transform NLP.
2020s	Diffusion models (e.g. Stable Diffusion) lead image and video generation.

Markov models

A **Markov chain** is a sequence of random states with the *Markov property*: the next state depends only on the current state, not on the earlier history, $P(s_{t+1} | s_t, s_{t-1}, \dots, s_1) = P(s_{t+1} | s_t)$. It is described by a **transition matrix** P with $P_{ij} = P(s_{t+1} = j | s_t = i)$, each row summing to 1. The distribution after k steps is $\pi_k = \pi_0 P^k$.

Example: Weather with states Sunny (S) and Rainy (R): $P = \begin{pmatrix} 0.8 & 0.2 \\ 0.4 & 0.6 \end{pmatrix}$.

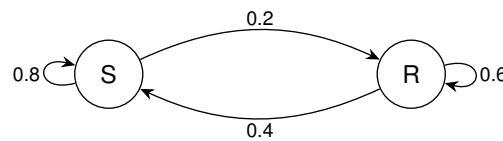


Figure 36: State diagram of the weather Markov chain.

Two step matrix: $P^2 = \begin{pmatrix} 0.72 & 0.28 \\ 0.56 & 0.44 \end{pmatrix}$ (for example $0.8(0.8) + 0.2(0.4) = 0.72$). If today is Sunny, $\pi_0 = (1, 0)$, the chance of Sunny two days later is 0.72. The **stationary distribution** $\pi = \pi P$ satisfies $0.2\pi_S = 0.4\pi_R$ with $\pi_S + \pi_R = 1$, so $\pi = (\frac{2}{3}, \frac{1}{3})$: in the long run it is sunny two days in three.

A **hidden Markov model** adds hidden states that emit observations; only the emissions are seen (used in speech recognition and tagging).

Generative adversarial networks (GAN)

Introduced by Goodfellow et al. (2014). A GAN is a two player zero sum game between two networks: the **generator** $G(\mathbf{z}; \theta_g)$ maps random noise $\mathbf{z} \sim p_z$ to fake data, and the **discriminator** $D(\mathbf{x}; \theta_d)$ outputs the probability in $[0, 1]$ that $\mathbf{x}$ is real. D learns to tell real from fake; G learns to fool D .

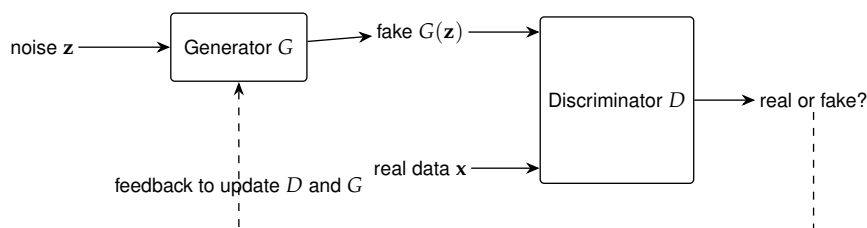


Figure 37: GAN training setup.

Minimax objective

$$\min_G \max_D V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_z} [\log(1 - D(G(\mathbf{z})))].$$

D maximises V (label real as 1 and fake as 0). G minimises it. For a fixed G the best discriminator is

$D^*(\mathbf{x}) = \frac{p_{\text{data}}(\mathbf{x})}{p_{\text{data}}(\mathbf{x}) + p_g(\mathbf{x})}$; at the optimum $p_g = p_{\text{data}}$ and $D = \frac{1}{2}$ everywhere (it can only guess).

Training alternates: update D on a batch of real and fake samples, then update G through D 's feedback. Known difficulties: unstable training and *mode collapse* (G produces only a few kinds of output). *Applications**: image to image translation, super resolution, medical image augmentation, synthetic data.

Knowledge Check: 40 Multiple Choice Questions

Choose one option per question. Answers are at the end. Questions 1 to 10 cover Chapter 2, 11 to 19 Chapter 3, 20 to 30 Chapter 4 and 31 to 40 Chapter 5.

Chapter 2: Problem Solving and Search

- Which of the following is *not* a component of a formal problem definition?

(A) Initial state	(B) A heuristic function
(C) Goal test	(D) Path cost
- An arc $X \rightarrow Y$ is arc consistent (AC3) when:

(A) every value of Y has some allowed value of X only	(B) X and Y have the same domain size
(C) all variables are already assigned	(D) every value of X has some allowed value of Y
- The minimum remaining values (MRV) heuristic chooses next the variable that:

(A) has the fewest legal values left	(B) has the most legal values left
(C) constrains the most unassigned variables only	(D) was assigned last
- Which uninformed search is complete and optimal for equal step costs but needs $O(b^d)$ memory?

(A) Depth first search	(B) Depth limited search
(C) Greedy best first search	(D) Breadth first search
- Iterative deepening search combines:

(A) the speed of greedy search with optimality of A*	(B) bidirectional search with heuristics
(C) hill climbing with random restarts	(D) the memory use of DFS with the completeness of BFS
- A heuristic $h(n)$ is admissible if it:

(A) never underestimates the true cost to the goal	(B) always equals the true cost to the goal
(C) never overestimates the true cost from n to the goal	(D) is always greater than $g(n)$
- Arad to Bucharest: greedy best first search finds a path of cost 450 while A* finds one of cost:

(A) 450	(B) 366
(C) 418	(D) 140
- A MAX root has three MIN children with leaves (3,5), (2,9), (0,7). With alpha beta pruning left to right, the root value and the number of leaves pruned are:

(A) 3 and 0	(B) 3 and 2
(C) 5 and 2	(D) 2 and 3
- In simulated annealing, as the temperature T falls, the probability $e^{-\Delta E/T}$ of accepting a worse move:

(A) decreases	(B) increases
(C) stays constant	(D) becomes exactly 0.5
- The main weakness of simple hill climbing is that it:

(A) always needs exponential memory	(B) cannot be applied to numeric problems
(C) never moves uphill	(D) can get stuck at a local maximum, a plateau or a ridge

Chapter 3: Knowledge Representation and Probabilistic Reasoning

11. In the Wumpus world, a breeze felt in a square indicates:

(A) the Wumpus in the same square	(B) gold in the same square
(C) a pit in the same square	(D) a pit in an adjacent square
12. Modus ponens infers Q from:

(A) $P \rightarrow Q$ and P	(B) $P \rightarrow Q$ and Q
(C) $P \rightarrow Q$ and $\neg Q$	(D) $P \vee Q$ and $\neg P$ only
13. Resolution refutation proves that $KB \models \alpha$ by:

(A) adding $\neg \alpha$ to the KB and deriving the empty clause	(B) adding α to the KB and deriving the empty clause
(C) finding a model where α is false	(D) removing all negations from the KB
14. Skolemisation is used to remove:

(A) universal quantifiers	(B) negation signs
(C) existential quantifiers	(D) implications
15. The sentence “all students study” is correctly written in first order logic as:

(A) $\forall x (Student(x) \wedge Studies(x))$	(B) $\exists x (Student(x) \rightarrow Studies(x))$
(C) $\forall x (Student(x) \rightarrow Studies(x))$	(D) $\exists x (Student(x) \wedge Studies(x))$
16. Forward chaining is best described as:

(A) goal driven, starting from the query	(B) a search using only negative facts
(C) data driven, starting from known facts and firing rules	(D) the same as backward chaining run twice
17. $P(spam) = 0.3$, $P(word|spam) = 0.6$, $P(word|ham) = 0.1$. Then $P(spam|word)$ is:

(A) 0.18	(B) 0.60
(C) 0.72	(D) 0.30
18. In the alarm Bayesian network with $B \rightarrow A \leftarrow E$, $A \rightarrow J$, $A \rightarrow M$, the joint probability is:

(A) $P(B)P(E)P(A)P(J)P(M)$	(B) $P(A B, E)P(J, M A)P(B E)$
(C) $P(J B)P(M E)P(A)$	(D) $P(B)P(E)P(A B, E)P(J A)P(M A)$
19. With fuzzy AND defined as the minimum, $\mu_A(x) = 0.6$ and $\mu_B(x) = 0.4$ give $\mu_{A \wedge B}(x) =$

(A) 0.6	(B) 0.24
(C) 1.0	(D) 0.4

Chapter 4: Machine Learning Fundamentals

20. In Mitchell’s definition, a program learns if its performance P on task T :

(A) improves with experience E	(B) is fixed regardless of E
(C) is measured only on the training data	(D) depends only on the model size
21. For $f(x) = x^2$, $x_0 = 3$ and learning rate $\eta = 0.25$, one gradient descent step gives $x_1 =$

(A) 0.75	(B) 1.5
(C) 2.5	(D) -1.5
22. The KKT complementary slackness condition for an inequality constraint $g_i(\mathbf{x}) \leq 0$ is:

(A) $\lambda_i g_i(\mathbf{x}) = 0$	(B) $\lambda_i < 0$
(C) $\nabla f = 0$ only	(D) $g_i(\mathbf{x}) > 0$
23. A model with very low training error but high test error is:

(A) underfitting	(B) perfectly generalised
(C) high bias and low variance	(D) overfitting

24. A coin shows 8 heads in 10 tosses. The maximum likelihood estimate of $P(\text{head})$ is:
 (A) 0.5 (B) 0.75
 (C) 0.8 (D) 0.2
25. ID3 chooses the split attribute with the:
 (A) lowest information gain (B) highest information gain
 (C) most distinct values (D) highest entropy after the split
26. An SVM has $\mathbf{w} = (3, 4)$. The margin width $2/\|\mathbf{w}\|$ is:
 (A) 2.5 (B) 0.4
 (C) 0.286 (D) 5
27. In Q learning with $\gamma = 0.8$, $r = 0$ and $\max_{a'} Q(s', a') = 100$, the target value $r + \gamma \max Q$ is:
 (A) 100 (B) 20
 (C) 80 (D) 180
28. A classifier has $TP = 40$, $FP = 10$, $FN = 5$, $TN = 45$. Its precision is:
 (A) 0.89 (B) 0.80
 (C) 0.85 (D) 0.50
29. In k fold cross validation each example is used for validation:
 (A) k times (B) exactly once
 (C) never (D) only in the first fold
30. Which of these is a generative classifier?
 (A) Naive Bayes (B) Logistic regression
 (C) Support vector machine (D) A perceptron

Chapter 5: Neural Networks and Deep Learning

31. The McCulloch and Pitts neuron fires when:
 (A) the output is negative (B) the weighted sum of binary inputs reaches the threshold
 (C) all weights are zero (D) the loss is minimum
32. The universal approximation theorem states that a feedforward network with:
 (A) one hidden layer, enough neurons and a non linear activation can approximate any continuous function (B) only linear activations can approximate any function
 (C) no hidden layer can learn XOR (D) exactly two neurons can learn any function
33. A single perceptron cannot learn XOR because XOR is:
 (A) a continuous function (B) a three input problem
 (C) too large for one neuron (D) not linearly separable
34. For a sigmoid output with binary cross entropy, $\hat{y} = 0.7$ and $y = 1$. The error signal $\delta = \hat{y} - y$ is:
 (A) 0.3 (B) 0.7
 (C) -0.3 (D) -0.7
35. Layer 1 has $n_1 = 4$ units and layer 2 has $n_2 = 2$ units. The shape of $W^{[2]}$ is:
 (A) 4×2 (B) 2×2
 (C) 2×4 (D) 4×4
36. A 7×7 input, 3×3 kernel, padding 1 and stride 2 give an output of size:
 (A) 4×4 (B) 3×3
 (C) 5×5 (D) 7×7

37. A convolution layer has 10 kernels of size 3×3 on a 3 channel input (with biases). Its parameter count is:
 (A) 270 (B) 90
 (C) 300 (D) 280
38. The LSTM cell state mainly helps to overcome:
 (A) lack of training data (B) the vanishing gradient over long sequences
 (C) the need for activation functions (D) overfitting in CNNs
39. For the weather chain $P = \begin{pmatrix} 0.8 & 0.2 \\ 0.4 & 0.6 \end{pmatrix}$, the probability Sunny is followed two days later by Sunny is:
 (A) 0.80 (B) 0.72
 (C) 0.64 (D) 0.28
40. At the optimum of a GAN the discriminator outputs:
 (A) $\frac{1}{2}$ for every input (B) 1 for every input
 (C) 0 for every input (D) $p_{\text{data}}(\mathbf{x})$

Answer Key

1	B	2	D	3	A	4	D	5	D
6	C	7	C	8	B	9	A	10	D
11	D	12	A	13	A	14	C	15	C
16	C	17	C	18	D	19	D	20	A
21	B	22	A	23	D	24	C	25	B
26	B	27	C	28	B	29	B	30	A
31	B	32	A	33	D	34	C	35	C
36	A	37	D	38	B	39	B	40	A